WNDRVR
SUPPORT NETWORK

VXWORKS 7 RADIUS AND DIAMETER PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Radius and Diameter Programmer's Guide, SR0650

16 September 2020

1. WIND RIVER RADIUS

1. Overview

1.1. Introduction

The Remote Authentication Dial-In User Service (RADIUS) is a client/server protocol for carrying authentication, authorization, and configuration messages between a network access point and a RADIUS server. Wind River supports the RADIUS client only as described in Part I of this guide.

Part II of this book describes the Diameter authentication, authorization, and accounting (AAA) protocol that provides support for peering AAA transactions across the Internet. Wind River supports the Diameter client, proxy agent, and relay agent as described in Part II of this guide.

Source Code Files

The source code files for RADIUS are in the following directory: *installDir* **/vxworks-7/pkgs_v2/net/ipnet/aaa-***x.x.x.x* **/radius**.

1.2. Diameter and RADIUS Differences

Diameter and RADIUS Differences tables on page 1 lists the major differences between the Diameter protocol and the RADIUS protocol.

Table 1. Diameter and RADIUS Differences

Item	Diameter	RADIUS
Agent support	distributed (Proxy, Redirect, Relay, and Translation)	implemented as part of the server does not support translation and redirect agents
Attribute data size	16,777,215 octets (32-bits)	255 octets (8-bits)
Extensibility	new commands and AVPs	not supported
Negotiation	negotiates supported applications and security	not supported
Peer discovery	static and dynamic	static
Security	hop-by-hop with Transport Layer Security (TLS) and end-to-end with IPsec	end-to-end
Server initiated message	supported (re-authentication, termination...)	not supported
Transport protocol	TCP and SCTP Wind River Diameter does not support Stream Control Transmission Protocol (SCTP) in this release.	UDP
Vendor-specific support	support for messages and AVPs	support for attributes

1.3. RADIUS Client

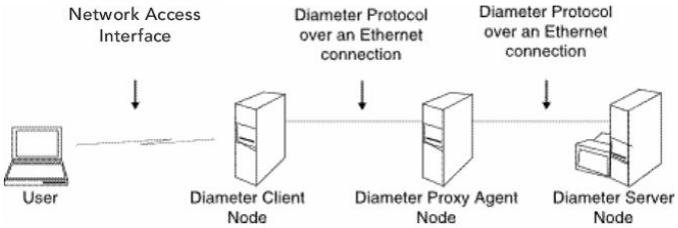
The Wind River RADIUS *client* is an IPv4/IPv6 capable API implementation of RADIUS. The RADIUS client acts as a framework library called by other Wind River products and applications. Any Wind River application that requires the RADIUS client can generate authentication traffic by calling the RADIUS APIs, after you set the necessary server configuration parameters.

The RADIUS server processes RADIUS client messages to grant or deny user access to network resources such as remote access servers, access concentrators, virtual private network (VPN) routers, and firewalls. The RADIUS server uses authentication schemes like PAP, CHAP, or EAP and a user database to verify the correctness of user-supplied information such as the network address, account status, and specific network service access privileges. The RADIUS server then returns one of three responses; Access Reject, Access Challenge, or Access Accept. An Access Challenge response from the RADIUS server is a request for additional information from the user such as a secondary password, PIN, or token. Access Challenge is also used in more complex authentication dialogs where a secure tunnel is established between the user device and the RADIUS Server. Once a user is logged in, the user is authorized to access various resources based on the rights and privileges assigned to the user account.

The RADIUS client runs on one or more network access points, sending authentication requests to a shared RADIUS server. The server databases contain user authentication and authorization data, as well as configuration data about the type of network access service provided, such as Point-to-Point Protocol (PPP), **rlogin**, **telnet**, and so on

RADIUS Client Example on page 2 figure depicts an authenticating client or user connecting to a Network Access Server (NAS) over a network connection. To authenticate the user, the NAS contacts a remote server running Internet Authentication Service (IAS). The NAS and the RADIUS server communicate using RADIUS.

Figure 1. RADIUS Client Example



A NAS operates as a client of a server. Servers that support the RADIUS protocol, are typically referred to as *RADIUS servers*. The RADIUS client, that is the NAS in this case, passes information about the user to a designated RADIUS server, and then acts on its response. To authenticate the user, the NAS sends an *authentication request* to the RADIUS server.

After a RADIUS server successfully authenticates a user, it returns configuration information to the NAS so that it can provide network service to the user. This configuration information is composed of *authorizations*.

The RADIUS server also collects various information sent by the NAS that can be used for accounting and for reporting on network activity. The RADIUS client sends information to designated RADIUS servers when the user logs on and logs off. The RADIUS client may send additional usage information on a periodic basis. The client sends *accounting requests* to the server to record logon/logoff and usage information.

While a RADIUS server processes the authentication request, it can perform authorization functions such as verifying the user's access information and checking whether the user already has a session in progress.

1.4. Deployment Options

You can deploy the RADIUS client in any of the following ways:

- Standalone

  The RADIUS client can request authentication on behalf of a user application using the RADIUS APIs.

- With Wind River PPP

  The RADIUS client can request authentication on behalf of Wind River PPP. In this deployment, the Wind River PPP generates authentication request packets and sends them to the designated RADIUS server; it also processes response packets received from the RADIUS server.

  For more information, see the VxWorks 7 PPP Programmer's Guide.

- With Wind River IKE

  The RADIUS client can request authentication on behalf of Wind River IKE. In this deployment, the Internet Key Exchange (IKE) daemon generates authentication request packets and sends them to the designated RADIUS server; it also processes response packets received from the RADIUS server.

  For more information, see the VxWorks 7 IKE Programmer's Guide.

RADIUS Client in Kernel Space

The RADIUS client must run in kernel space and all applications using the client must also run in kernel space. There is no support to run the RADIUS client from user-space.

Note: You cannot access the RADIUS APIs from real-time processes (RTPs).

1.5. RFCs

Wind River RADIUS complies with the IETF RFCs listed in this section.

- RFC 2865: **Remote Authentication Dial In User Service (RADIUS)**
- RFC 2866: **RADIUS Accounting**
- RFC 2868: **RADIUS Attributes for Tunnel Protocol Support**
- RFC 2869: **RADIUS Extension (section 2.3.x - EAP)**
- RFC 3162: **RADIUS and IPv6**
- RFC 4372: **Chargeable User Identity**
- RFC 4668: **RADIUS Authentication Client MIB**
- RFC 4670: **RADIUS Accounting Client MIB**
- RFC 4675: **RADIUS Attributes for Virtual LAN and priority support**
- RFC 5247: **Extensible Authentication Protocol (EAP) Key Management Framework**, section 5.9

Note: Wind River RADIUS is a framework library that customers can use to achieve RFC 2865 and RFC 2868 support for their applications. For example, Wind River RADIUS supports authentication for telnet server customers who login using telnet. Depending on a customer's desired setup, further development is required to implement features of RFC 2865 and RFC 2868, such as rlogin, and tunnel protocol support, respectively.

2. Configuration and Build

2.1. Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VxWorks Source Build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VxWorks Image Project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source configuration options for RADIUS, see Source Build Configuration Options (VSB) on page 4.

For a list of available components for RADIUS, see Kernel Configuration Components (VIP) on page 4.

For a list of available configuration parameters for RADIUS, see RADIUS Configuration Parameters on page 5.

2.2. Source Build Configuration Options (VSB)

RADIUS VSB Options table on page 4 lists the RADIUS VSB option.

Table 1. RADIUS VSB Options

Name	Description
IPNET_RADIUS	Enables the remote authentication dial-in user service (RADIUS) protocol library.

2.3. Kernel Configuration Components (VIP)

RADIUS VIP Components table on page 4 lists optional components for your application.

Table 1. RADIUS VIP Components

Name	Description
INCLUDE_IPRADIUS	Include the RADIUS client in your project.

2.4. Optional Kernel Configuration Components

Optional Components table on page 4 lists optional components for your application.

Table 1. Optional Components

Name	Description
INCLUDE_IPATTACH	Attaches the shared memory END device to the IPv4 stack. The device is attached to all supported network-layer protocols built into the stack. See the VxWorks 7 Network Stack Programmer's Guide.
INCLUDE_IP6ATTACH	Attaches the shared memory END device to the IPv6 stack. The device is attached to all supported network-layer protocols built into the stack. See the VxWorks 7 Network Stack Programmer's Guide
INCLUDE_IFCONFIG	Include support for the configuration of the kernel-resident network interfaces.
INCLUDE_IPD_CMD	Include the daemon control shell command (ipd).
INCLUDE_IPRADIUS_CMD	Include the RADIUS client shell command (radiusc).
INCLUDE_IPCOM_SYSLOGD_CMD	Include the system logging shell command (syslog).

Name	Description
INCLUDE_IPCOM_SYSVAR_CMD	Include the system variables shell command (sysvar).
INCLUDE_PING	Ping tool to locate network devices across an IP network.

2.5. RADIUS Configuration Parameters

RADIUS Client Configuration Parameter table on page 5 lists the RADIUS configuration parameter and system variable.

Table 1. RADIUS Client Configuration Parameter

Parameter Name, System Variable Name, and Description	Default and Type
RADIUS_NAS_IDENTIFIER ipradius.nas.identifier Identifies the Network Access Server (NAS) that originates the authentication or accounting request and must be set if an IP address is unavailable.	**"Wind River example NAS"** char *

2.6. Accessing the RADIUS MIBs with the MIB Browser Tool

To access the RADIUS Management Information Base (MIB) using the MIB browser tool, you must configure your VSB and VIP with the components required by Wind River SNMP and MIB support. For more information, see the *Wind River SNMP for VxWorks 7 Programmer's Guide*.

3. Shell Commands

3.1. Introduction

The RADIUS shell command (radiusc) is the easiest way to become familiar with the RADIUS client. The radiusc shell command allows you to send a RADIUS access request to a RADIUS server, access the MIB parameters, initiate an accounting session, manage RADIUS server entries, and manage the RADIUS client state.

You can use the ipd shell command to view the state of the RADIUS client service (ipradius).

You can use the sysvar shell command to modify RADIUS configuration parameters (ipradius). RADIUS Client Configuration Parameter table on page 5 correlates RADIUS VIP configuration parameters to their respective system variables.

You can use the **syslog** shell command to set the priority for the RADIUS syslog debug messages (**ipradius**). For more information, see VxWorks 7 Network Stack Programmer's Guide.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C
->
```

3.2. RADIUS Client Shell Command

To access the radiusc shell command mode, enter the following command at the kernel shell prompt:

```
-> cmd radiusc
radiusc>
```

To return to the kernel shell, enter the following command:

```
radiusc> exit
->
```

Table 1 on page 6 lists the Diameter client shell commands in alphabetical order and provides a description of each command

Table 1. RADIUS Client Shell Commands

Shell Command	Description
access -user *user* -pwd *password* [-servip *ipAddress*] [-servip2 *ipAddress*] [-pwdtype *type*] [-tmosec *timeout*] [-pollnum *pollRequests*] [-pollsec *pollInterval*] [-cui *chargeableUserIdentity*]	Sends a RADIUS access request to a RADIUS server to test the client to server interface. The **user** and pwd parameter values are case sensitive. The servip modifier defines the first server to send a request to. The servip2 modifier defines a second server to send a request to if the first server is unavailable. The password type (pwdtype) modifier can be set to 0 (no password), 1 (PAP) or 2 (CHAP). The default pwdtype is 2 (PAP). The timeout (tmosec) modifier sets the maximum time to wait for a server response. The default tmosec is 30 seconds. The number of request attempts (pollnum) modifier sets the number of access requests to send to a server before declaring an error. The default pollnum is set to 3 requests. The poll interval (pollsec) modifier sets the interval between failed access requests to a server. The default pollsec is set to 3 seconds. The pollnum and pollsec modifier settings also apply to the acct and servadd shell commands. The cui modifier sets the chargeable user identity for accounting purposes. The options are none or a string of 3 or more characters. The tmosec modifier setting also applies to the acct shell command.
accessmib	Lists the access MIB parameters and their values as defined in RFC 2618, **RADIUS Authentication Client MIB**.
accounting -user *user* -id *identifier* [-servip *ipAddress*] [-servip2 *ipAddress*] [-intnum *interimPolls*] [-intsec *intervalSeconds*] [-tmosec *timeout*] [-pollnum *pollRequests*] [-pollsec *pollInterval*]	Initiates an accounting session. The default user handle identifier (id) is 0. The servip modifier defines the first server to send a request to. The servip2 modifier defines a second server to send a request to if the first server is unavailable. The number of interim accounting polls (intnum) modifier sets the number of attempts to access a server before switching to the next server. The default intnum value is set to 0. The interim accounting interval (intsec) modifier sets the interval between accounting sessions. The minimum intsec value is 3 seconds and the default value is set to 5 seconds.

Shell Command	Description
	The timeout (tmosec) modifier sets the maximum time to wait for a server response. The default tmosec value is set to 30 seconds. The number of request attempts (pollnum) modifier sets the number of failed consecutive accounting requests to send to a server before declaring an error. The default pollnum value is set to 3 requests. The poll interval (pollsec) modifier sets the interval for accounting requests. The maximum pollsec value is 600 seconds and the default value is set to 3 seconds. The pollnum and pollsec modifier settings also apply to the access and servadd shell commands. The tmosec modifier setting also applies to the access shell command.
acctmib	Lists the accounting MIB parameters and their values as defined in RFC 2620, **RADIUS Accounting Client MIB**.
close	Closes the RADIUS client globally. All requests and accounting sessions are terminated and all server entries are deleted.
exit	Exits the RADIUS client command shell. RADIUS configuration data and running data are preserved.
help	Displays the RADIUS client shell commands and modifiers.
open	Opens the RADIUS client globally. You only need to use this command if you closed the RADIUS server with the radiusc close command. On startup, the RADIUS client is opened.
servadd -servip *ipAddress* -secret *secret* [-port *port*] [-servtype *type*] [-pollnum *pollRequests*] [-pollsec *pollInterval*]	Adds a RADIUS server entry. The server is queried when requests for access or accounting are made. The secret parameter is case sensitive. The default port is 1812. The server type (servtype) options are 1 (access), 2 (account), and 3 (full). The default servtype is set to 3 (full server). The number of request attempts (pollnum) modifier sets the number of failed consecutive accounting requests to send to a server before declaring an error. The default pollnum value is set to 3 requests. The poll interval (pollsec) modifier sets the interval for accounting requests. The maximum pollsec value is 600 seconds and the default value is set to 3 seconds. The pollnum and pollsec modifier settings also apply to the acct and access shell commands.
servdel -servip *ipAddress* -port *port*	Deletes a RADIUS server entry. Closing the RADIUS client with the close command deletes all server entries.

Using the RADIUS Client Shell Command on page 7

Using the RADIUS Client Shell Command

Getting Help

To display help for the radiusc commands, enter **help**.

```
radiusc> help
```

Opening and Closing the RADIUS Client

The RADIUS client automatically starts upon booting the target. The following example shows how to close and reopen the RADIUS client. Once the RADIUS client is closed, all requests and accounting sessions are terminated and all server entries are deleted.

```
radiusc> close
close
closed radius
radiusc>
```

```
radiusc> open
open
opened radius
radiusc>
```

Managing RADIUS Server Entries

The servadd command adds a RADIUS server entry. The servdel command removes a RADIUS server entry. Use the accessmib or acctmib command to view the server entries. The following example shows how to add 2 servers, view the entries, and delete a server entry.

```
radiusc> servadd -servip 128.224.150.240 -port 8820 -secret abcDEF
servadd -servip 128.224.150.240 -port 8820 -secret abcDEF
added radius server entry

radiusc> servadd -servip 128.224.150.241 -port 8880 -secret abcDEF
servadd -servip 128.224.150.241 -port 8880 -secret abcDEF
added radius server entry

radiusc> accessmib
accessmib
RFC 2618 - Radius Authentication Client MIB:
-------------------------------------------
radiusAuthClientInvalidServerAddresses = 0
radiusAuthClientIdentifier = Interpeak example NAS

Server #1:
radiusAuthServerIndex = 5
radiusAuthServerAddress = 128.224.150.240
radiusAuthClientServerPortNumber = 8820
radiusAuthClientRoundTripTime = 0
radiusAuthClientAccessRequests = 0
radiusAuthClientAccessRetransmissions = 0
radiusAuthClientAccessAccepts = 0
radiusAuthClientAccessRejects = 0
radiusAuthClientAccessChallenges = 0
radiusAuthClientMalformedAccessResponses = 0
```

```
radiusAuthClientBadAuthenticators = 0
radiusAuthClientPendingRequests = 0
radiusAuthClientTimeouts = 0
radiusAuthClientUnknownTypes = 0
radiusAuthClientPacketsDropped = 0

Server #2:
radiusAuthServerIndex = 6
radiusAuthServerAddress = 128.224.150.241
radiusAuthClientServerPortNumber = 8880
radiusAuthClientRoundTripTime = 0
radiusAuthClientAccessRequests = 0
radiusAuthClientAccessRetransmissions = 0
radiusAuthClientAccessAccepts = 0
radiusAuthClientAccessRejects = 0
radiusAuthClientAccessChallenges = 0
radiusAuthClientMalformedAccessResponses = 0
radiusAuthClientBadAuthenticators = 0
radiusAuthClientPendingRequests = 0
radiusAuthClientTimeouts = 0
radiusAuthClientUnknownTypes = 0
radiusAuthClientPacketsDropped = 0

radiusc> servdel -servip 128.224.150.241 -port 8880
servdel -servip 128.224.150.241 -port 8880
removed radius server entry

radiusc> accessmib
accessmib
RFC 2618 - Radius Authentication Client MIB:
--------------------------------------------
radiusAuthClientInvalidServerAddresses = 0
radiusAuthClientIdentifier = Interpeak example NAS

Server #1:
radiusAuthServerIndex = 5
radiusAuthServerAddress = 128.224.150.240
radiusAuthClientServerPortNumber = 8820
radiusAuthClientRoundTripTime = 0
radiusAuthClientAccessRequests = 0
radiusAuthClientAccessRetransmissions = 0
radiusAuthClientAccessAccepts = 0
radiusAuthClientAccessRejects = 0
radiusAuthClientAccessChallenges = 0
radiusAuthClientMalformedAccessResponses = 0
radiusAuthClientBadAuthenticators = 0
radiusAuthClientPendingRequests = 0
radiusAuthClientTimeouts = 0
radiusAuthClientUnknownTypes = 0
radiusAuthClientPacketsDropped = 0
```

Viewing the Access and Accounting MIB Parameters

The following example shows how to list the access MIB parameters and their values as defined in RFC 2618, **RADIUS Authentication Client MIB**.

```
radiusc> accessmib
accessmib
RFC 2618 - Radius Authentication Client MIB:
-------------------------------------------
radiusAuthClientInvalidServerAddresses = 0
radiusAuthClientIdentifier = Interpeak example NAS

Server #1:
radiusAuthServerIndex = 8
radiusAuthServerAddress = 128.224.150.240
radiusAuthClientServerPortNumber = 8820
radiusAuthClientRoundTripTime = 0
radiusAuthClientAccessRequests = 0
radiusAuthClientAccessRetransmissions = 0
radiusAuthClientAccessAccepts = 0
radiusAuthClientAccessRejects = 0
radiusAuthClientAccessChallenges = 0
radiusAuthClientMalformedAccessResponses = 0
radiusAuthClientBadAuthenticators = 0
radiusAuthClientPendingRequests = 0
radiusAuthClientTimeouts = 0
radiusAuthClientUnknownTypes = 0
radiusAuthClientPacketsDropped = 0
```

The following example shows how to list the accounting MIB parameters and their values as defined RFC 2620, **RADIUS Accounting Client MIB**.

```
radiusc> acctmib
acctmib
RFC 2620 - Radius Accounting Client MIB:
-----------------------------------------
radiusAccClientInvalidServerAddresses = 0
radiusAccClientIdentifier = Interpeak example NAS

Server #1:
radiusAccServerIndex = 5
radiusAccServerAddress = 128.224.150.240
radiusAccClientServerPortNumber = 8821
radiusAccClientRoundTripTime = 0
radiusAccClientRequests = 9
radiusAccClientRetransmissions = 18
radiusAccClientResponses = 0
radiusAccClientMalformedResponses = 0
radiusAccClientBadAuthenticators = 0
radiusAccClientPendingRequests = 0
radiusAccClientTimeouts = 9
radiusAccClientUnknownTypes = 0
radiusAccClientPacketsDropped = 0
```

Accessing a RADIUS Server

Once the client and servers are configured with the appropriate usernames, passwords, and secrets, the client to server interface can be tested with the access command. The access command sends a RADIUS access request to a RADIUS server to test the client to server interface. One or two RADIUS servers can be specified. The following example shows how to send an access request to a server and a backup server (servip2). The command also specifies a set of parameters for the interface. The server responds with either valid (successful access request) or invalid (failed access request).

```
radiusc> access -user rbudd -pwd ABCdef -servip 128.224.150.240 -servip2
128.224.150.241 -pwdtype 1 -tmosec 6 -pollnum 4
access -user rbudd -pwd rbudd -servip 128.224.150.240 -servip2 128.224.150.241
-pwdtype 1 -tmos
ec 6 -pollnum 4
sending access request(s)...
<- ipradius_event_access_accept, id = 0 [hdr 6]
User 'rbudd' access: valid
```

Requesting Accounting Information from the RADIUS Server

Once the client to server interface is operational, use the **accounting** command to initiate an accounting session and obtain accounting information from the server. The accounting server coordinates must be entered in the server entry. The following example shows how to add a server entry for the accounting port, view the new entry, and initiate an accounting session to the server.

```
radiusc> servadd -servip 128.224.150.240 -secret rbudd -port 8821
added radius server entry
radiusc> acctmib
acctmib
RFC 2620 - Radius Accounting Client MIB:
-------------------------------------------
radiusAccClientInvalidServerAddresses = 0
radiusAccClientIdentifier = Interpeak example NAS

Server #1:
radiusAccServerIndex = 5
radiusAccServerAddress = 128.224.150.240
radiusAccClientServerPortNumber = 8821
radiusAccClientRoundTripTime = 0
radiusAccClientRequests = 9
radiusAccClientRetransmissions = 18
radiusAccClientResponses = 0
radiusAccClientMalformedResponses = 0
radiusAccClientBadAuthenticators = 0
radiusAccClientPendingRequests = 0
radiusAccClientTimeouts = 9
radiusAccClientUnknownTypes = 0
radiusAccClientPacketsDropped = 0

radiusc> accounting -user rbudd -servip 128.224.150.240 -id 1
accounting -user rbudd -servip 128.224.150.240 -id 1
starting accounting for id = 1
<- ipradius_event_accounting_request, id = 1 [hdr 0]
<- ipradius_event_accounting_response, id = 1 [hdr 0]
started accounting for id = 1

stopping accounting for id = 1
<- ipradius_event_accounting_request, id = 1 [hdr 0]
<- ipradius_event_accounting_response, id = 1 [hdr 1]
stopped accounting for id = 1
```

Typical RADIUS Session

The following example creates a RADIUS session to send an access request and initiate an accounting session. This example adds a RADIUS server with an IP address of 10.1.2.1 with a secret of kallekula123. It then sends an access request to that server for user myname and password mypassword. Finally, accounting information is requested from the accounting server at port 1813.

```
-> cmd radiusc
radiusc> servadd -servip 10.1.2.1 -secret kallekula123
radiusc> accessmib
radiusc> access -user myname -pwd mypassword
radiusc> servadd -servip 10.1.2.1 -secret kallekula123 -port 1813
radiusc> acctmib
radiusc> accounting -user myname -id 1
radiusc> exit
```

Using this example, if a RADIUS server is running on 10.1.2.1 and it shares the same *secret* as the client, you should see whether *myname* can login or not and receive accounting information.

Parent topic: RADIUS Client Shell Command on page 6

4. API Functions
4.1. RADIUS Client API Functions

The RADIUS client APIs consist of public functions that can be called from the host shell or from an application, after you load and start the RADIUS client. This section briefly describes these functions and notes important caveats. For information on the API functions, see the **ipradius** API reference entries. The API functions are defined in **ipradius.c**. For implementation details, see Applications on page 16.

The RADIUS client interface functions perform the following tasks:

- Locate an available RADIUS server.
- Manage the interface to a RADIUS server.
- Submit an access-request to a RADIUS server.
- Manage the attribute list of an access-request.
- Get properties of an access-request attribute.

Server Location Routines

The ipradius_ctrl( ) function is used to get or set global RADIUS client information or configuration. The default behavior of the RADIUS client is to always send interim accounting requests as well as the accounting stop requests to the server which initially replied on the accounting start request.

You have the option of disabling this behavior using ipradius_ctrl( ), however, Wind River advises you to be cautious and prepare for this development, perhaps with back-up source code and project files, should you choose to disable this behavior.

Server Interface Routines

The functions below manage the interface to a specific RADIUS server.

ipradius_get_numattempts( )

This function gets the number of times a request is sent to a server.

ipradius_accounting_session_start( )

This function starts the accounting session.

ipradius_accounting_session_stop( )

This function stops the accounting session.

ipradius_check_request( )

This function validates that RADIUS packet attributes are correct, and so on, by checking for a pending request (access or accounting) with the unique ID.

ipradius_read( )

This function parses an incoming RADIUS server reply. It is called when a packet from the RADIUS server is available. The RADIUS client parses the RADIUS packet (specified by buf and len) and acts on it. The calling function must provide a pointer to a socket address structure (from_addr) from where the received packet came. Note, the application is responsible for listening on the RADIUS client socket supplied in the request structure, and passing all input to ipradius_read( ).

ipradius_server_add( )

This function adds a new server to the server list. The RADIUS client queries all servers with the requested service, accounting or authentication. Any number of servers may be entered. You may specify a given server to handle access, accounting or both. New servers added first in the server list, are chosen before old servers.

The RADIUS client picks the first server of the matching type (access or account) and uses that one. If that one does not answer, it picks the next one. Alternatively, a desired list of ordered servers may be specified for a specific access or accounting request. Note, servers must still be added by use of this function even if listed in a request structure because required server information, such as the server secret, can only be set by adding a server.

A server is uniquely identified by its socket address (with family, length, IP address and port), type, pollnum and pollsec. If an identical server is added, a reference counter is increased. To fully remove a server, an application must call ipradius_server_remove( ) the same amount of times that it was added.

This function can also be used to create a RADIUS client IPv6 server, specifying the retransmission data and the shared secret. You must ensure that the server does not already exist in the server list. To avoid a memory leak, you must explicitly free the handle when no longer needed.

ipradius_server_ctrl( )

This function gets or sets server specific RADIUS client information or configuration. The server is identified by its socket address structure server->server_addr (family, length, address and port) or the server index alone server->indexGet server statistics and information.

ipradius_server_remove( )

This function removes a server from the server list and the server is no longer queried for authentication and accounting. A server is uniquely identified by its socket address structure (with family, length, IP address, and port), type, pollnum and pollsec or by its index alone. If there are multiple references to a server (that is, it has been added multiple times) the reference counter is decreased by one. When the server reference counter is reduced to zero, the entry is finally removed). Hence, to fully remove a server, an application must call this function the same number of times as it was added by ipradius_server_add( ).

request->event_callback( )

This input and output callback is for RADIUS request events. It is called by the RADIUS client code and valid for any of the following events:

- incoming RADIUS server reply packets
- access and accounting request timeouts
- cancelled requests
- each new accounting request sent

The event code contains the event that caused the callback to be called. For more information on event codes, see the event_callback( ) API reference entry.

Access-Request Submission Routine

ipradius_access_request sends a RADIUS client access request to the RADIUS server. If the specified server does not respond, it submits a second request to any available server that can handle the request. This process continues until either a server responds or no more servers are available. This function terminates if a server responds but rejects the request.

Prior to calling this function, ipradius_access_request_pkt_malloc( ) must be called to get a RADIUS packet.

The *request* must point to an **Ipradius_request** structure which is filled in by the application. The entire structure is copied by the RADIUS client and returned in the callbacks. The request structure contains both variables which benefits the application and variables which are of interest to the RADIUS client.

Access-Request Attribute List Routines

The access-request attribute list functions manage the attribute list of an access-request.

ipradius_access_request_pkt_free( )

This function releases the entire packet (including attributes) and frees a RADIUS access request packet that was passed to the ipradius_access_request( ) function.

ipradius_access_request_pkt_malloc( )

This function allocates and initializes an entire RADIUS access request packet including space for attributes. After all the application attributes are added, this function sends the access request by calling ipradius_access_request( ). Access request RADIUS attributes must be added after calling this function and prior to calling ipradius_access_request( ).

ipradius_attr_add_array( )

This function adds the desired byte array attribute to the specified attribute list. The attribute is pointed to by data of length len, to the RADIUS packet pointed to by packet. The attribute type is specified by type.

ipradius_attr_add_u32( )

This function adds the desired 32-bit attribute data to the RADIUS packet pointed to by packet. The attribute data is converted from host to network byte order by the function.

ipradius_attr_change_u32( )

This function replaces attribute type value with the 32-bit value data in packet pointed to by packet. Attribute length is always six. A 32-bit (**Ip_u32**) attribute appears as:

```
attribute type | total attribute length | b0 | b1 | b2 | b3
```

b0-b3 is the 32 bit value in network byte format. The total length is 6, (1+1+4) since the attribute type and total attribute length fields are both 1 byte in size.

ipradius_attr_get_array( )

This function gets an array of data from the first attribute of the requested type. The function returns IP_NULL( ) if attribute was not present in the packet.

ipradius_attr_get_u32( )

This function gets the 32-bit value of the first attribute in the packet of the requested type and stores it in host byte order in the variable pointed to by data.

ipradius_attr_get_next(pkt,0)

This function gets a pointer to the value of the next attribute in the packet pointed to by **pkt**.

ipradius_attr_get_next(pkt, current attr)

This function finds the attribute type in the header. The attribute value is then found in the attribute header.

Access-Request Attribute Routines

The access-request attribute functions get properties of the current attribute in the specified attribute list.

ipradius_attr_find( )

This function returns a pointer to the value of the designated attribute, for example the pointer points to the first attribute data byte after the length field. The function returns **IP_NULL** if the attribute was not present in the packet. Note, if there is more than one attribute of the same type, the first of those is returned.

ipradius_attr_get_next(pkt, 0)

This function retrieves the current attribute. The attribute type and/or length is then found in the attribute header.

ipradius_attr_get_next(pkt, current attr)

This function finds the attribute type in the header. The attribute value is then found in the attribute header.

RADIUS APIs table on page 15 lists and describes configuration APIs with defaults as installed.

Table 1. RADIUS APIs

Name	Description	Type	Default
Shell Command	leave uncommented to register the RADIUS client shell commands	**ipcom_shell_add_cmd( )** function installing ipradius_cmd_radiusc	Registered
Access Attribute Size	maximum size of access request attribute field	IPRADIUS macro **IPRADIUS_ access_ request_attribute_size**	2048
Accounting Attribute Size	maximum size of accounting request attribute field	IPRADIUS macro **IPRADIUS_accounting _request_attribute_size**	500

Name	Description	Type	Default
System Priority	build (initial) system priority used with IPCOM syslogs. Set to **IPCOM_LOG_ NONE** for no syslogs	IPRADIUS macro I**PRADIUS_SYSTEM_ PRIORITY**	`#if defined(_WRS_CONFIG_IPRADIUS_SYSLOG_PRIORITY) #define IPRADIUS_SYSTEM_PRIORITY _WRS_CONFIG_IPRADIUS_SYSLOG_PRIORITY #else #define IPRADIUS_SYSTEM_PRIORITY IPCOM_LOG_WARNING #endif`
Maximum Poll Delay	maximum RADIUS request poll delay in seconds	IPRADIUS macro **IPRADIUS_MAX_POLL_ DELAY**	600
Minimum Accounting Interval	minimum accounting interval in seconds	IPRADIUS macro **IPRADIUS_MIN_ACCT_ INTERVAL**	60

5. Applications

5.1. Introduction

The RADIUS client manages authentication and accounting information for a VxWorks application running on a network access point device. It enables the application to communicate with a set of alternate RADIUS servers running standards-compliant RADIUS server software.

Wind River RADIUS is intended for use in a wide range of embedded applications and therefore makes no assumptions on the types of attributes an application may append on a packet sent to the server.

The RADIUS client enables the transfer of accounting information to a central server. Accounting may be started or stopped by the use of two start and stop functions. An *access request* is also generated by the use of two functions, one to create the packet, and one to send it. Prior to sending the access request packet, the application may add any number of attributes to the packet through the use of attribute add functions. The RADIUS client uses the chargeable user identity (CUI) attribute to support roaming transactions that occur outside of the home network.

All RADIUS server responses such as a received accept, reject or response, or a request timeout due to a server failure, result in an event callback. The application has access to the accounting request packets with the use of callbacks before they are sent. Application attributes may be added using callbacks. For more information, see Request Event Callbacks on page 19.

5.2. System Description

The RADIUS client is an application framework of function calls that execute in the context by which they are called. The functions are protected from race conditions using an internal RADIUS code mutex.

Since the RADIUS client uses global variable names, there can only be one copy of the RADIUS client in a single module, even with many users (applications) of the library. The RADIUS client can handle any number of multiple servers/multiple session-transactions.

5.3. Interaction with RADIUS Servers

An application calls ipradius_server_add( ) to add new servers, and conversely ipradius_server_remove( ) to remove servers. The application defines how a server processes authentication requests, accounting requests, or both.

One RADIUS client can handle any number of RADIUS servers. The RADIUS client validates servers with matching properties beginning with the last added server. Alternatively, a desired and ordered list of servers may be specified for a specific access or accounting request.

Note: Even servers listed in a request must still be added globally because required server information, such as the server secret for example, can only be set by adding a server.

If a server is removed, no more requests are sent to it. The same applies to responses to a request from a server before it is removed; they are also silently discarded.

The RADIUS client can have up to 256 outstanding requests of the same type (accounting and access) per unique server IP address. After this, new access requests or accounting sessions must wait until the server(s) process more pending access/accounting requests. This processing is due to the 8-bit identifier field in the RADIUS packet header.

5.4. The Authentication Process

In the authentication process, requests are outgoing, or sent from the RADIUS client to the RADIUS server. Accepts, rejects, and responses are incoming, or sent from the RADIUS server to the RADIUS client.

The authentication process typically involves the following steps:

1. The RADIUS client obtains authentication data from the user attempting to access the network and creates an access-request containing attributes such as user name, user password, client ID, chargeable user identity (CUI), and port ID to which the user is attempting access.
2. If a password is included in the authentication data, the RADIUS client hashes it using a method based on the RSA MD5 message digest algorithm.
3. The RADIUS client submits the access-request over the network to the RADIUS server. If there is no response within a specified length of time, the client resubmits the request for a specified number of times.
4. If initial submissions fail, the client can forward requests to one or more alternate servers.
5. If the client receives an *access-challenge* from a server, it executes a user-supplied callback function. It is up to the writer of this function to decide whether to issue another request with a new request ID, in which the response replaces the user-password attribute.

Authentication Transactions

An access-request/access-accept is a transaction between the client and the server. After the access-accept is received the transaction is over, and the RADIUS client discards the request and all other data from the transaction. However, an application may keep track of several transactions.

An access-request – access-accept/access-reject transaction does not leave any saved state or data within RADIUS. From the perspective of the RADIUS client, nothing connects an access-request/access-accept transaction with a particular accounting session. However, an application may connect them by using the same unique identifier.

If an application requires interim accounting, the accounting interval must be saved by the application from the access-accept packet and then provided in the call to ipradius_accounting_session_start( ). For more information on this function, see the API reference entry.

Access Requests

The application calls ipradius_access_request( ) to send a request packet to the server.

Before calling ipradius_access_request( ), however, the application should add a Password Authentication Protocol (PAP) or Challenge-Handshake Authentication Protocol (CHAP) password to the request structure.

RADIUS encrypts the password and adds a password attribute to the request packet. When using CHAP with no cleartext access to the password, as is the case with the PPP protocol, the user must add the **IPRADIUS_ATTR_CHAP_Password** attribute with the CHAP ID and response.

Before the packet is sent to the RADIUS server, the application must get the request packet by calling ipradius_access_request_packet_get( ) and then add all additional attributes required. The application can add any number of attributes of any type prior to calling ipradius_access_request( ) as long as they fit in the packet.

The client does not reject unknown attributes. The RADIUS client provides a comprehensive toolbox of functions to inspect attributes in packets and to add or change attributes.

If the RADIUS server is satisfied with the request, it sends an access-accept to the RADIUS client.

If an access-accept lacks valid authentication or has faulty length attributes, it is silently discarded. However, the application is notified using the return code from ipradius_read( ). An application may log errors of this type as desired.

If the RADIUS server is not satisfied with the request, it sends an access-reject packet. The reply message attribute should contain a message detailing the reason for the reject.

5.5. Accounting Sessions

The accounting session process typically involves the following steps:

1. The application begins an accounting session by calling ipradius_accounting_session_start( ).
2. The RADIUS client prepares an *accounting request* packet and calls the request->event_callback(**IPRADIUS_EVENT_ACCOUNTING_REQUEST**, …) function, that allows the application to optionally add any number of attributes to the packet.
3. After the event callback, the client sends the request to the server. If interim accounting is requested, the RADIUS client sends accounting updates with the requested frequency. For each successful RADIUS server accounting response, the request- >event_callback(**IPRADIUS_EVENT_ACCOUNTING_RESPONSE**, …) is called, with pointers to access the request and response packet as well as the server that responded.
4. The application ends the accounting session with a call to ipradius_accounting_session_stop( ).

The default behavior of the RADIUS client is to always send interim accounting requests as well as the accounting stop request to the server which initially replied to the accounting start request. This ensures that all records of a particular session reside on the same server, therefore simplifying the work of the accounting software that uses the records.

The RADIUS client can also operate in a mode where any server gets the accounting requests. This provides the greatest likelihood that no accounting records are lost. The application can select the desired operating mode by using ipradius_ctrl( ).

Only the available memory and communication throughput limits the number of simultaneous accounting sessions.

If the server does not respond to the accounting requests, the RADIUS client continues to send requests until all available servers are queried for the application-specified number of times. If all servers fail, the session is automatically removed by the RADIUS client since it was never started. This occurrence is indicated by request>event_callback(**IPRADIUS_EVENT_ACCOUNTING_TIMEOUT**, …) being called with the response packet and server set to **IP_NULL**.

Unique Handles

The application should set a unique handle (a 32-bit ID), provided in the request structure to the RADIUS client for new authentication requests, and accounting sessions. The unique value is used to match responses to the request to which they respond. They also prevent identical requests from being sent.

The unique handle can be any 32-bit value. The only requirement is that different requests of the same type should not have the same 32-bit value. The RADIUS client neither knows nor cares how the application creates the unique values.

5.6. Request Event Callbacks

The RADIUS client uses the event callback function as a general mechanism to:

- let users add attributes to outgoing accounting packets
- notify applications of server reply packets
- notify applications of timeout or other failure events

Each access request or accounting request may use its own event callback.

The application supplies a 32-bit cookie in the request structure. The cookie can be accessed in the event callback function from the request structure. The cookie is not read or written, but typically used to find request specific application defined data in the event callback.

For more than 32 bits of data, you should use ipcom_malloc( ) to obtain a buffer to contain a data structure, from which the cookie can be used to point to the buffer.

The only request callback event is for accounting, because interim accounting requires request attributes to be changed for each new accounting request. Even if the accounting request packet has to be sent several times and/or perhaps to several servers, the request event callback is only called once, prior to the first copy of the new request that is sent to the first server.

The following example includes a call to ipradius_access_request( ) and implements a callback function. For more information, see **ipradius_cmd_radiusc.c**. For more information about event codes, see Server Interface Routines on page 12.

```
IP_STATIC Ip_err
ipradius_example_event_callback(int event, Ipradius_request *request,
Ipradius_header *request_packet, Ipradius_header *response_packet,
Ipradius_server *server)
{
    Vars *vars;
    Ip_u8 *data;
    Ip_u32 type;
    Ip_err retval;
    Ip_u8 flen;
    int len;

    (void)request_packet;
    (void)response_packet;
    (void)server;

    vars = request->cookie;
    switch(event)
    {
    case IPRADIUS_EVENT_ACCESS_ACCEPT:
        vars->valid_user = TRUE;
        ip_assert(request->status == IPCOM_SUCCESS);
        len = ipcom_sprintf(vars->buf, "<- ipradius_event_access_accept,
```

```
        id =%ld [hdr %d]"IP_LF,request->unique, (int)response_packet->id);

        /* check attributes here. */

        /* Print attributes. */
        for (data = ipradius_attr_get_next(response_packet, NULL);
            data != NULL && len < (IPRADIUS_BUF_SIZE - 25);
            data = ipradius_attr_get_next(response_packet, data))
        {
            if (data[0] == IPRADIUS_ATTR_CUI) /* CUI */
        {
            len += ipcom_sprintf(vars->buf + len, "attr #%3d len %3d cui-value
            %s"IP_LF, data[0], data[1], (char *)&data[2]);
        }   else
        {
            len += ipcom_sprintf(vars->buf + len, "attr #%3d len %3d"IP_LF,
            data[0], data[1]);
        }
        }

        ipcom_printf(vars->buf);
        break;

    case IPRADIUS_EVENT_ACCESS_REJECT:
        ip_assert(request->status == IPCOM_ERR_FAILED);
        ipcom_printf("<- ipradius_event_access_reject, id = %ld [hdr
        %d]"IP_LF,request->unique, (int)response_packet->id);
        break;

    case IPRADIUS_EVENT_ACCOUNTING_RESPONSE:
        ip_assert(request->status == IPCOM_SUCCESS);
        len = ipcom_sprintf(vars->buf, "<-ipradius_event_accounting_response,
        id = %ld [hdr %d]"IP_LF,request->unique, (int)response_packet->id);
        vars->intfound++;

         for (data = ipradius_attr_get_next(response_packet, NULL);
             data != NULL && len < (IPRADIUS_BUF_SIZE - 25);
             data = ipradius_attr_get_next(response_packet, data))
         {
             if (data[0] == IPRADIUS_ATTR_CUI) /* CUI */
         {
             len += ipcom_sprintf(vars->buf + len, "attr #%3d len %3d cui-value
             %s"IP_LF, data[0], data[1], (char *)&data[2]);
         }   else
         {

             len += ipcom_sprintf(vars->buf + len, "attr #%3d len %3d"IP_LF,
             data[0], data[1]);
         }
         }

         ipcom_printf(vars->buf);
         break;

     case IPRADIUS_EVENT_ACCOUNTING_REQUEST:
         if (vars->pid == ipcom_getpid())
             ipcom_printf("<- ipradius_event_accounting_request, id = %ld
```

```
            [hdr %d]"IP_LF, request->unique,(int)request_packet->id);

        /* Add type specific accounting variables. */
        retval = ipradius_attr_get_u32(request_packet,
        IPRADIUS_ATTR_Acct_Status_Type, &type);
        if (retval == IPCOM_SUCCESS)
        {
            switch(type)
            {
            case IPRADIUS_ACCT_Start:
                if (*vars->user && ipradius_attr_find(request_packet,
                IPRADIUS_ATTR_User_Name, &flen) == NULL)
                (void)ipradius_attr_add_array(request_packet,
                IPRADIUS_ATTR_User_Name, (Ip_u8*)vars->user,
                (Ip_u8)ipcom_strlen(vars->user));
                break;
             case IPRADIUS_ACCT_Stop:
                 break;
             default:
                 break;
             }
         }

         /* Add general missing accounting attributes. */
         if (ipradius_attr_find(request_packet,
             IPRADIUS_ATTR_Acct_Session_Id, &flen) == NULL)
            (void)ipradius_attr_add_u32(request_packet,
            IPRADIUS_ATTR_Acct_Session_Id, request->unique);
        break;

      case IPRADIUS_EVENT_ACCESS_TIMEOUT:
            if (vars->pid == ipcom_getpid())
            ipcom_printf("<- ipradius_event_access_timeout, id = %ld"IP_LF,
            request->unique);
        break;
      case IPRADIUS_EVENT_ACCESS_CANCELLED:
        if (vars->pid == ipcom_getpid())
            ipcom_printf("<- ipradius_event_access_cancelled, id =
            %ld"IP_LF, request->unique);
        break;
      case IPRADIUS_EVENT_ACCOUNTING_TIMEOUT:
        if (vars->pid == ipcom_getpid())
            ipcom_printf("<- ipradius_event_accounting_timeout,
            id = %ld"IP_LF, request->unique);
        break;
      case IPRADIUS_EVENT_ACCOUNTING_CANCELLED:
        if (vars->pid == ipcom_getpid())
            ipcom_printf("<- ipradius_event_accounting_cancelled,
            id = %ld"IP_LF, request->unique);
        break;

    default:
        IP_PANIC();
        break;
    }
    return IPCOM_SUCCESS;
}
```

5.7. RADIUS Shell Command Application

Table 1 on page 22 lists the API function calls used in the RADIUS client shell command application located in **ipradius_cmd_radiusc.c**. You can use this implementation as a baseline for your application.

Table 1. Example API Calls from the RADIUS Source Files

API Routine	Calling Routine
ipradius_attr_add_array( )	**ipradius_example_event_callback( )** **ipradius_cmd_access( )**
ipradius_attr_add_u32( )	**ipradius_example_event_callback( )**
ipradius_attr_find( )	**ipradius_example_event_callback( )**
ipradius_attr_get_next( )	**ipradius_example_event_callback( )**
ipradius_attr_get_u32( )	**ipradius_example_event_callback( )**
ipradius_access_request( )	**ipradius_cmd_access( )**
ipradius_access_request_pkt_malloc( )	**ipradius_cmd_access( )**
ipradius_accounting_session_startipradius_accounting_session_start( )	**ipradius_cmd_accounting( )**
ipradius_accounting_session_stop( )	**ipradius_cmd_accounting( )**
ipradius_close( )	**ipradius_cmd_close( )**
ipradius_ctrl( )	**ipradius_cmd_accessmib( )**
ipradius_open( )	**ipradius_cmd_open( )**
ipradius_read( )	**ipradius_example_recv( )**
ipradius_server_add( )	**ipradius_cmd_servadd( )**
ipradius_server_ctrl( )	**ipradius_cmd_accessmib( )**
ipradius_server_remove( )	**ipradius_server_remove( )**

2. WIND RIVER DIAMETER

1. Overview

1.1. Introduction

The Diameter authentication, authorization, and accounting (AAA) protocol provides support for peering AAA transactions across the Internet. The Diameter base protocol provides the minimum requirements needed for AAA protocol and remote network access applications. Diameter runs on one or more network access points, sending authentication requests to a Diameter server. The server databases contain user authentication and authorization data, as well as configuration data defining the available network access services.

The Diameter protocol has several advantages over previous AAA protocols in that it offers improvements in the areas of reliability, security, scalability, and flexibility.

The Diameter base protocol provides the following functionality:

- connection and session management
- user authentication and capabilities negotiation
- reliable delivery of Attribute Value Pairs (AVPs)
- agent support for proxy, redirect, and relay servers
- extensibility, through the addition of new commands and AVPs
- basic accounting services
- dynamic peer discovery

Wind River Diameter supports the creation and configuration of client, proxy agent, and relay agent modules on all VxWorks-supported targets. These modules can interface with all RFC-compliant Diameter nodes in a network. Wind River Diameter now supports the Diameter Base Protocol as specified by **draft-ietf-dime-rfc3588bis-17.txt**. The Diameter Base Protocol brings the following changes to Wind River Diameter:

- Each Diameter packet contains a single application ID AVP.

  The application ID in a Abort-Session-Answer/Abort-Session-Request, Session-Termination -Request/Session-Termination-Answer, Re-Auth-Answer/Re-Auth-Request contain the application ID associated with the session.
- The events of receiving a Capability-Exchange-Request at an initiating node and receiving a Capability-Exchange-Answer at a responding node are removed from the peer finite state machine (FSM).
- The **DIAMETER_COMMAND_UNSUPPORTED**, **DIAMETER_INVALID_HDR_BITS**, **DIAMETER_INVALID_AVP_BITS**, and **DIAMETER_UNKNOWN_PEER** have been redefined and moved from the protocol error messages to the permanent failure messages.
- **The DIAMETER_INVALID_BIT_IN_HEADER** and **DIAMETER_INVALID_MESSAGE_LENGTH** have been redefined and moved from the permanent failure messages to the protocol error messages.

  Dynamic peer discovery is disabled by default. To enable dynamic peer discovery, see Configuring Dynamic Peer Discovery on page 32.

  Diameter clients support server initiated re-authentication requests for a given session.

Source Code Files

The source code files for Diameter are in the following directory: *installDir/* **vxworks-7/pkgs_v2/net/ipnet/aaa-***x.x.x.x* **/diameter**.

1.2. Diameter and RADIUS Differences

Diameter and RADIUS Differences on page 24 table lists the major differences between the Diameter protocol and the RADIUS protocol.

Table 1. Diameter and RADIUS Differences

Item	Diameter	RADIUS
Agent support	distributed (Proxy, Redirect, Relay, and Translation)	implemented as part of the server does not support translation and redirect agents
Attribute data size	16,777,215 octets (32-bits)	255 octets (8-bits)
Extensibility	new commands and AVPs	not supported
Negotiation	negotiates supported applications and security	not supported
Peer discovery	static and dynamic	static
Security	hop-by-hop with Transport Layer Security (TLS) and end-to-end with IPsec	end-to-end
Server initiated message	supported (re-authentication, termination...)	not supported
Transport protocol	TCP and SCTP Wind River Diameter does not support Stream Control Transmission Protocol (SCTP) in this release.	UDP
Vendor-specific support	support for messages and AVPs	support for attributes

1.3. Diameter Tutorial

The tutorial in Diameter Tutorial on page 72 describes the implementation of an example Diameter network environment with a Diameter client, a Diameter proxy agent, and an Open Diameter authentication server. The tutorial guides you through the creation and configuration of a Diameter client for a Core target and a proxy agent for a PowerPC target. The access client shell command tests the Diameter protocol interface between the client and the proxy agent. The openaccess client shell command simulates Diameter sessions linking the client to the server via the proxy agent.

1.4. RFCs

Wind River Diameter complies with the IETF RFCs listed in this section.

- RFC 3539: *Authentication, Authorization and Accounting (AAA) Transport Profile*
  - Transport Mappings, section 3.1 (no SCTP)
  - Multiple Connections, section 3.3
  - Application Layer Watchdog, section 3.4 (except section 3.4.3 Connection Load Balancing)
  - Duplicate Detection, section 3.5
  - Premature Failover, section 3.10

— Security Considerations on TLS and IPsec support, section 4

- RFC 3588: *The Diameter Base Protocol* (except specifics related to redirect and translation agents, servers, SCTP, and client accounting)
- *Diameter Base Protocol draft-ietf-dime-rfc3588bis-17.txt*
- RFC 4005: *Diameter Network Access Server Application*
- RFC 4072: *Diameter Extensible Authentication Protocol (EAP) Application*
- RFC 4675 *RADIUS Attributes for Virtual LAN and priority support*

  Information on Open Diameter is accessible from the following Web site:

  http://diameter.sourceforge.net/

2. Diameter Basics

2.1. Introduction

This chapter provides an overview of the following Diameter topics:

- Diameter network components
- Elements of a Diameter session
- Diameter peer interaction
- The authentication process
- The routing processes

2.2. Diameter Overview

Diameter manages authentication information for a VxWorks application running on a network access point device. It enables the application to communicate with a set of Diameter peers running standards-compliant Diameter server software.

Diameter is intended for use in a wide range of embedded applications and therefore makes no assumptions on the types of Attribute-Value-Pairs (AVPs) an application may append on a packet sent to the server.

All Diameter server responses such as a *received accept* , *reject* , *response* , or a *request time out* due to a server failure, result in an event callback. The application, with the use of callbacks, has access to authentication request packets.

Diameter is an enhanced version of the RADIUS AAA protocol and improves the protocol in several areas including:

- connection and session management
- peer dynamic discovery and capabilities negotiation
- reliable transport
- agent support
- extensibility

Diameter applications can define their own commands and AVPs to implement enhancements. A translation agent provides interoperability between a RADIUS network and a Diameter network.

The Diameter API provides IPv4/IPv6 functions that support IETF standards for authentication and security applications that relate to remote networks. Designed as a peer-to-peer architecture, each node can operate as an agent, a client, or a server. The Diameter agent can be a relay agent, a proxy agent, a redirect agent, or a translation agent. A relay agent forwards messages to the specified destinations. A proxy agent can operate as a relay agent and provide value-added services, enforce rules, and perform administrative tasks. A redirect agent processes messages and provides redirection information to the sender, and maintains a peer table with a list of Diameter nodes and a peer routing table.

To enhance reliability, Diameter connections and sessions use Transmission Control Protocol (TCP). Diameter messages convey commands and notifications to Diameter nodes. The AVPs convey routing, security, and capability information to Diameter nodes.

Diameter supports the detection of protocol errors and application errors. Protocol error messages highlight problems with the underlying protocol used to carry Diameter messages. Application error messages highlight problems with the Diameter protocol itself.

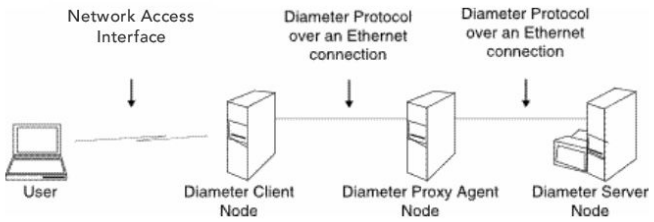
The Diameter API allows applications to call Diameter functions as part of a running task. Once the necessary peer configuration parameters are set, any application that requires Diameter can generate authentication traffic by calling the Diameter API functions.

2.3. Diameter Network Components

The Example Diameter Network Environment on page 26 figure below details the basic components of a Diameter network. They include the following:

- **Diameter node**: A host that implements the Diameter protocol, and acts either as a client, an agent, or a server.
- **Diameter user**: An entity requesting access to resources by means of the Diameter base protocol.
- **Diameter client**: A device at the edge of the network that performs access control. An example of a Diameter client is a Network Access Server (NAS).
- **Diameter relay agent**: An application that forwards requests and responses based on routing-related Attribute-Value-Pairs (AVPs) and realm routing table entries. Relay agents do not make policy decisions and do not originate messages. Relay agents are capable of handling any Diameter application or message type. Because relay agents make decisions based on information in routing AVPs and realm forwarding tables, they do not keep state information on NAS resource usage or sessions in progress.
- **Diameter proxy agent**: In addition to being a relay agent, a proxy agent also makes policy decisions related to resource usage and provisioning. This is typically accomplished by tracking the state of NAS devices. While proxy agents typically do not respond to client requests prior to receiving a response from the server, they may originate reject messages in cases where policies are violated. As a result, proxy agents need to understand the semantics of the messages passing through them, and may not support all Diameter applications. The main task for the proxy agent is to receive and parse incoming Diameter packets. Packets are either forwarded, discarded (duplicate packets), or rejected (unauthorized packets).
- **Diameter server**: A Diameter server is one that handles authentication, authorization, and accounting requests for a particular realm. By its very nature, a Diameter server must support Diameter applications and the Diameter base protocol.
- **Connection**: A transport level link between two adjacent nodes (peers) used to send and receive Diameter messages.
- **Diameter protocol**: A networking framework to provide Authentication, Authorization, and Accounting (AAA) services for applications such as network access or IP mobility. Diameter is also intended to work in both local and roaming environments.

Figure 2. Example Diameter Network Environment



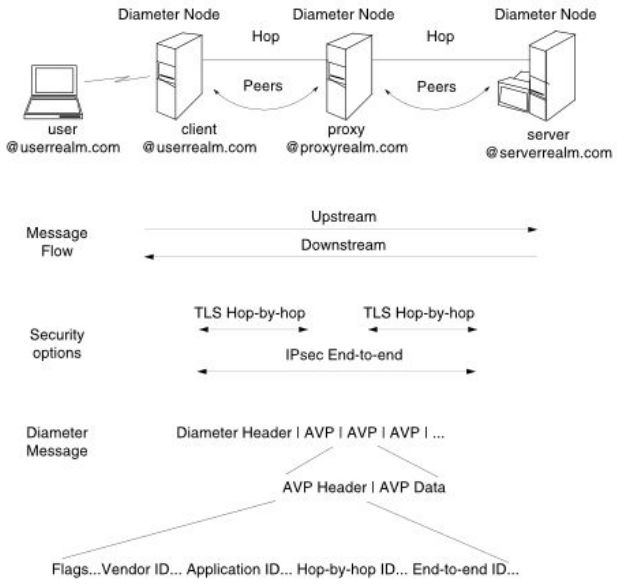
2.4. Diameter Session

A Diameter session is an application layer exchange of commands and AVPs between an authorized Diameter client and a Diameter server over one or more connections. TheElements of a Diameter Session on page 27 figure details the elements of a Diameter session. They include the following:

- **Diameter peer**: A Diameter node to which another Diameter node has a direct transport connection.

- **Realm**: The string in the Network Access Identifier (NAI) that immediately follows the @ character. Configured host IDs must follow NAI format as defined in RFC 4282. In Diameter, the realm is used for message routing purposes.
- **Downstream**: Identifies the direction of a particular Diameter message from the home server towards the access device.
- **Upstream**: Identifies the direction of a particular Diameter message from the access device towards the home server.

  Security: Transport security for a session is either provided by TLS (hop-by-hop) or IPsec (end-to-end).
- **Attribute-Value-Pair (AVP)**: The Diameter protocol consists of a header followed by one or more Attribute-Value-Pairs (AVPs). Each AVP consists of a header and protocol-specific data such as routing, authentication, authorization, or accounting information.
- **Vendor identifier**: A numerical value assigned to a vendor of Diameter applications.
- **Application identifier**: A numerical value that identifies a protocol based on Diameter. The application can be an authentication application, an accounting application, or a vendor specific application.
- **Hop**: A span linking two Diameter peers.



2.5. Interaction with Peers

Diameter requires transmission level security for each connection (TLS or IPsec). Therefore, each connection is authenticated, replay and integrity protected, and confidential on a per-packet basis.

Communication between Diameter peers begins with one peer sending a message to another Diameter peer. The Diameter application defines the set of AVPs in a message.

The initial request for authentication and/or authorization of a user must include a Session-ID. The Session-ID is then used in all subsequent messages to identify the user's session. The communicating party may accept the request, or reject it by returning an answer message with the Result-Code AVP set to indicate an error. The specific behavior of the Diameter server or client receiving a request depends on the Diameter application.

Wind River Diameter Capabilities on page 28 Wind River Diameter supports the creation and configuration of client, proxy agent, and relay agent modules on all VxWorks-supported targets. These modules can interface with all RFC-compliant Diameter nodes in a network.

Wind River Diameter Capabilities

Wind River Diameter supports the creation and configuration of client, proxy agent, and relay agent modules on all VxWorks-supported targets. These modules can interface with all RFC-compliant Diameter nodes in a network.

One Diameter client can communicate with any number of Diameter servers.

Parent topic: Interaction with Peers on page 27

2.6. Authentication Process

In the authentication process, requests are outgoing. That is, they are sent from the Diameter client to the Diameter server or Diameter agent. Accepts, rejects, and responses are incoming, or sent from the Diameter server or Diameter agent to the Diameter client.

The authentication process typically involves the following steps:

1. The Diameter client obtains authentication data from the user attempting to access the network. The user creates an access-request containing AVPs such as the session identifier, application identifier, password, event callback function, and the server coordinates the user is attempting access. Diameter clients can use TLS or IPsec to protect the Diameter packets.
2. The Diameter client submits the access-request over the network to the Diameter server. If there is no response within a specified length of time, the client resubmits the request a specified number of times.
3. If initial submissions fail, the client can forward requests to one or more alternate servers.
4. If the client receives an access-challenge from a server, it executes a user-supplied callback function. It is up to the application to issue a new request ID.

A server may initiate a re-auth-request (RAR) for a particular session. The client responds with a re-auth-answer (RAA) to state whether the server-initiated re-authentication is supported. If re-authentication is supported, a service specific authentication access request follows. If re-authentication is not supported, the Diameter client disconnects the device and closes the session. A **IPDIAMETER_EVENT_RAR** event is generated when the Diameter client receives a RAR.

2.7. Using TLS to Protect Diameter Packets

The Transport Layer Security (TLS) protocol is used to protect Diameter packets exchanged on a single hop connection between Diameter peers. Using TLS prevents violation of the connection security by a man-in-the-middle attack. This protection involves encryption of the TLS protocol traffic as well as the Diameter protocol traffic. The lowest layer clearly visible in the packets is the transport layer, for example, TCP.

The encryption and decryption is done using a self-signed certificate from a commercial Certificate Authority (CA), a user certificate that was created by the CA by digitally signing a Certificate Signing Request (CSR), and a user key that was used to generate the CSR.

Establishing a TLS tunnel between 2 peers requires that both peers have access to certificates and the related private key. These certificates are available from the commercial Certificate Authorities. See Openssl Example on page 89 for a description of these files.

Using the secure key database feature together with TLS prevents violation of the connection security by an insider attack. It does so by importing the private key used for the encryption of the connection traffic into the secure key database using a pass phrase known only to the administrator. Without the use of the secure key database the pass phrase is stored as clear text on the client or proxy node, making it available to a malicious insider.

During encryption and decryption operations the key is retrieved from the secure key database using a predefined keyID, and the pass phrase is not required.

See Configuring TLS on page 39 for a description of the build time configuration of TLS.

2.8. Event Callbacks

The Diameter client uses the event callback function as a general mechanism to:

- notify applications of server reply packets
- notify applications of timeout or other failure events

Only the event callback function submitted in the first request structure can be used for requests of a given session identifier.

The application supplies a 32-bit cookie in the request structure. The cookie can be accessed in the event callback function from the request structure. The cookie is not read or written, but is typically used to find request-specific application data in the event callback.

For data items larger than 32 bits, use ipcom_malloc( ) to obtain a buffer to contain the data structure, from which the cookie can be used to point to the buffer.

For more information, see the example ipdiameter_example_event_callback( ) function in the **ipdiameter_cmd_diameterc.c** source file and Event Callback on page 67. For information about event codes (**IPDIAMETER_EVENT_***), see the **ipdiameter_constant.h** source file.

2.9. Error Messages

Nodes at each hop process protocol error messages.

The supported protocol error messages are:

- DIAMETER_UNABLE_TO_DELIVER 3002

  This error indicates that Diameter cannot deliver a message to the next hop.
- DIAMETER_LOOP_DETECTED 3005

  This error indicates that an agent has detected a loop while trying to get a message to the intended recipient. The error message is sent to the client.

Nodes issue permanent error messages to inform a peer that the request failed, and should not be attempted again. The error message is sent to the client. The supported permanent error message is:

- DIAMETER_AUTHORIZATION_REJECTED 5003

  This error indicates that a request is received for which the user could not be authorized.

2.10. Routing Process

Diameter makes use of **policy enforcement lists**, **peer tables**, and **realm routing tables** to forward Diameter packets between peers. The routing process makes use of information in the AVPs to determine the destination of each packet.

Policy enforcement list: As part of policy enforcement, proxy agents use a set of unauthorized user names or hosts. If a packet is associated with an item in the policy enforcement list, the packet is discarded and a response message with a permanent failure code of **DIAMETER_AUTHORIZATION_REJECTED** is forwarded to the downstream peer. You use the policyadd, policyshow, and policydel proxy agent shell commands to manage the Policy Enforcement List entries.

Peer table: Diameter clients and agents use lookup tables of Diameter peers to implement message forwarding. If a peer is not found in the peer table, a Diameter client discards the packet, while a Diameter agent routes the packet to the realm routing process. When a packet is discarded, a reply message with a protocol error failure code of **DIAMETER_UNABLE_TO_DELIVER** is forwarded to the downstream peer. You manage **peer table entries with the peeradd**, **peershow**, and **peerdel shell commands**.

Realm routing table: Agents use lookup tables of realm-based routes, and optionally, a default route to implement message forwarding. If a route is not found in the realm routing table, the packet is discarded and a reply message with a protocol error failure code of **DIAMETER_UNABLE_TO_DELIVER** is forwarded to the downstream peer.

You manage realm routing table entries with the **routeadd**, **routeshow**, and **routedel** agent shell commands.

Client Routing Process

The Wind River Diameter client uses a peer table to route Diameter packets between a user and a peer. The peer routing process searches the peer table to locate a peer for the next hop. If a peer is found, the packet is routed to the peer. If a peer is not found, the packet is discarded.

Proxy Agent Routing Process

The Wind River Diameter proxy agent uses a policy enforcement list, a peer table, and a realm routing table to process Diameter packets. The policy enforcement process searches the policy enforcement list to determine if the user or the realm is listed. If a match is not found by the policy enforcement process, the peer routing process is invoked to search the peer table for a peer. If a peer is found in the peer routing table, the packet is routed to the peer. If a peer is not found in the peer routing table, the realm routing process is invoked to search the realm routing table to locate a route to the next hop. If a route is found in the realm routing table, the packet is routed to the next hop. If a route is not found in the realm routing table and a default route exists, the packet is routed to the next hop. If a default route is not found, the packet is discarded.

Relay Agent Routing Process

The Wind River Diameter relay agent uses a peer table and a realm routing table to process Diameter packets. The peer routing process is invoked to search the peer table for a peer. If a peer is found in the peer routing table, the packet is routed to the peer. If a peer is not found in the peer routing table, the realm routing process is invoked to search the realm routing table to locate a route to the next hop. If a route is found in the realm routing table, the packet is routed to the next hop. If a route is not found in the realm routing table and a default route exists, the packet is routed to the next hop. If a default route is not found, the packet is discarded.

3. Configuration and Build

3.1. Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VxWorks Source Build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VxWorks Image Project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source configuration options for Diameter, see Source Build Configuration Options (VSB) on page 31. Additional VSB options and source file configuration options are described in the following sections:

- Optional VSB Configuration Options on page 31
- Configuring the Diameter Debug Options on page 31
- Configuring Dynamic Peer Discovery on page 32
- Enabling the Diameter Client Shell Test Command on page 33
- Enabling the Testing of Diameter EAP on page 33

For a list of available Diameter components, see Kernel Configuration Components (VIP) on page 33.

For a list of Diameter configuration parameters, see Diameter Client Configuration Parameters on page 37 and Diameter Client Configuration ParametersDiameter Proxy Agent Configuration Parameters on page 35.

3.2. Source Build Configuration Options (VSB)

The Diameter VSB Options on page 31 table lists the Diameter VSB options.

Table 1. Diameter VSB Options

Name	Description
IPNET_DIAMETER	Enable Diameter in the VSB.
COMPONENT_IPDIAMETERC	Enable the Diameter client in the VSB.
COMPONENT_IPDIAMETERP	Enable the Diameter proxy agent in the VSB.

3.3. Optional VSB Configuration Options

The Optional Diameter VSB Options on page 31 table lists optional VSB security options for your Diameter application.

Table 1. Optional Diameter VSB Options

Name	Description
IPDIAMETER_SEC_KEY_DB	Enable secure key database support for Diameter components. For implementation details, see Configuring TLS on page 39.
IPNET_EAP	Enable the extensible authentication protocol (EAP). See also, Enabling the Testing of Diameter EAP on page 33. For more information see the
IPNET_IPSECIKE	Enable support for IP security and internet key exchange. For more information see the VxWorks 7 IPsec Programmer's Guideand VxWorks 7 IKE Programmer's Guide.
OPENSSL	Enable support for the secure socket library. For more information see the VxWorks 7 SSL Programmer's Guide.

Configuring the Diameter Debug Options on page 31
Configuring Dynamic Peer Discovery on page 32
Enabling the Diameter Client Shell Test Command on page 33 To enable the testing of node connections to a Diameter server from the Diameter client shell, you must change #undef **OPEN_DIAMETER_TEST** to #define **OPEN_DIAMETER_TEST** in the following files:
Enabling the Testing of Diameter EAP on page 33 To enable the testing of Diameter EAP with the openaccess command, change #undef **OPEN_DIAMETER_EAP** to #define **OPEN_DIAMETER_EAP** in **ipdiameter_cmd_diameterc.c**.

Configuring the Diameter Debug Options

To enable debug messages you enable the debug macro in the source files by changing **#undef DIAMETER_DEBUG** to **#define DIAMETER_DEBUG**. The Source Files to Edit for Debugging on page 32 table lists the source files and their corresponding debug modes. After changing the state of the debug mode, you must rebuild the VSB, rebuild the VxWorks image, and reload the image into the target.

Table 1. Source Files to Edit for Debugging

Edit the following file ...	Edit the following file ...
ipdiameter.c	the basic operation of Diameter.
ipdiameter_client.c	the diameter client.
ipdiameter_client_auth_session_fsm.c	an authentication session.
ipdiameter_cmd_diameterc.c	the execution of client shell commands.
ipdiameter_cmd_diameterp.c	the execution of agent shell commands.
ipdiameter_eap.c	diameter EAP.
ipdiameter_pagent.c	the diameter agent.
ipdiameter_peer_fsm.c	peer state transitions.
ipdiameter_peer_mgnt.c	peer management.
ipdiameter_routing.c	diameter agent routing.
ipdiameter_rx.c	packet reception.
ipdiameter_session_mgnt.c	session management.
ipdiameter_tls.c	transport Layer Security.
ipdiameter_tx.c	packet transmission.
ipdiameter_util.c	internal Diameter utilities.

Note: Enabling debug options adds printf commands to the code that can change the timing behavior and introduce delays that may impact functionality.

Parent topic: Optional VSB Configuration Options on page 31

Configuring Dynamic Peer Discovery

To change the state of dynamic peer discovery, you set the state and rebuild the VSB and VIP. To view the current state of dynamic peer discovery, access **ipdiameter_config.h** and locate **IPDIAMETER_DYNAMIC_PEER_DISCOVERY**. The default setting disables dynamic peer discovery as follows:

```
/* Macro for dynamic peer discovery feature */

#undef IPDIAMETER_DYNAMIC_PEER_DISCOVERY
```

To enable dynamic peer discovery, change #undef to #define as follows:

```
/* Macro for dynamic peer discovery feature */
#define IPDIAMETER_DYNAMIC_PEER_DISCOVERY
```

To implement the change, rebuild the VSB and VIP.

Parent topic: Optional VSB Configuration Options on page 31

Enabling the Diameter Client Shell Test Command

To enable the testing of node connections to a Diameter server from the Diameter client shell, you must change #undef **OPEN_DIAMETER_TEST** to #define **OPEN_DIAMETER_TEST** in the following files:

- **ipdiameter_client.c**
- **ipdiameter_cmd_diameterc.c**
- **ipdiameter_rx.c**
- **ipdiameter_tx.c**

These changes enable the openaccess shell command and allow you to initiate any of the following tests from the Diameter client shell:

- Single session testing
- Multi session testing
- Extensible Authentication Protocol testing

For example usage of the openaccess command as part of the Diameter tutorial in Diameter Tutorial on page 72, see Testing the Diameter Protocol Interface to the Server on page 84.

Parent topic: Optional VSB Configuration Options on page 31

Enabling the Testing of Diameter EAP

To enable the testing of Diameter EAP with the openaccess command, change #undef **OPEN_DIAMETER_EAP** to #define **OPEN_DIAMETER_EAP** in **ipdiameter_cmd_diameterc.c**.

For example usage of the openaccess command to test Diameter EAP, see Testing the Diameter Protocol Interface to the Server on page 87.

Parent topic: Optional VSB Configuration Options on page 31

3.4. Kernel Configuration Components (VIP)

The Diameter VIP Components on page 34 table lists the required Diameter VIP components for Diameter functionality. In a single image file, you can include support for a Diameter client or a Diameter proxy agent, but not both.

Table 1. Diameter VIP Components

Name	Description
INCLUDE_IPDIAMETERC	Include support for the Diameter client. Automatically includes **INCLUDE_IPSSL** and **INCLUDE_IPDIAMETER_TLS_CFG**.
INCLUDE_IPDIAMETERP	Include support for the Diameter proxy agent. Automatically includes **INCLUDE_IPSSL** and **INCLUDE_IPDIAMETER_TLS_CFG**.
INCLUDE_IPDIAMETER_EAP	Include the Diameter extensible authentication protocol (EAP). See Enabling the Testing of Diameter EAP on page 33. To configure EAP components and parameters, see the .
INCLUDE_IPDIAMETER_TLS_CFG	Include the Diameter transport layer security (TLS) configuration parameters to support hop-to-hop security with TLS.

3.5. Optional Kernel Configuration Components

The Optional Components on page 34 table lists optional components for your Diameter application

Table 1. Table 8-5 : Optional Components

Name	Description
INCLUDE_IPATTACH	Attaches the shared memory END device to the IPv4 stack. The device is attached to all supported network-layer protocols built into the stack. See the VxWorks 7 Network Stack Programmer's Guide.
INCLUDE_IP6ATTACH	Attaches the shared memory END device to the IPv6 stack. The device is attached to all supported network-layer protocols built into the stack. See the VxWorks 7 Network Stack Programmer's Guide.
INCLUDE_IFCONFIG	Include support for the configuration of the kernel-resident network interfaces.
INCLUDE_IPCOM_FS_NATIVE	Include support for importing a private key file into the secure key database from a remote host. See Configuring TLS on page 39.
INCLUDE_IPCOM_KEY_DB_CMD	Include support for importing a private key file from a remote host using the key database shell command (**keydb**). See Importing a Key File for a TLS Connection on page 63.

Name	Description
INCLUDE_IPDIAMETERC_CMD	Include support for the Diameter client shell command (**diameterc**). See Enabling the Diameter Client Shell Test Command on page 33.
INCLUDE_IPDIAMETERP_CMD	Include support for the Diameter proxy agent shell command (**diameterp**). See Enabling the Diameter Client Shell Test Command on page 33.
INCLUDE_IPD_CMD	Include the daemon control shell command (**ipd**).
INCLUDE_IPCOM_SYSLOGD_CMD	Include the system logging shell command (**syslog**).
INCLUDE_IPCOM_SYSVAR_CMD	Include the system variables shell command (**sysvar**).
INCLUDE_PING	Include the Ping tool to locate network devices across an IP network.

3.6. Diameter Client Configuration Parameters

The Diameter Client Configuration Parameters on page 35 table lists the Diameter client configuration parameters and system variables.

Table 1. Diameter Client Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_ASR_COMPLY ipdiameter.asrcomply Sets the response to an **Abort-Session-Request** command. Set to 1 to terminate the session upon receiving an abort request or set to 0 to ignore the abort request.	**"1"** char *
DIAMETER_APPLICATION_LIST ipdiameter.appl_list Defines the list of application identifiers supported by the Diameter local node. The format is a comma-separated list of application identifiers. The options are Diameter NASREQ (1), Diameter Base Accounting (3), Diameter Credit Control (4), or **Diameter EAP(5)**.	**"5"** char *
DIAMETER_AUTH_GRACE_PERIOD ipdiameter.timeout.authgraceperiod Specifies the number of seconds the Diameter server waits following the expiration of the Authorization-Lifetime AVP before cleaning up resources for the session.	**"30"** char *
DIAMETER_AUTH_LIFETIME ipdiameter.timeout.authlifetime Specifies the maximum number of seconds of service to be provided to the user before the user is to be re-authenticated and/or re-authorized.	**"360"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_HOST_DOMAIN ipdiameter.domain Diameter local node address family. Set to **2** for IPv4 or **28** for IPv6.	**"2"** char *
DIAMETER_IDENTIFIER ipdiameter.identifier Diameter entity identity, which is the node's fully qualified domain name (FQDN). This identity string is used to get the node's address with **getaddrinfo( )**. The user must enter a valid and resolvable entry for this string.	**"apple. example.com"** char *
DIAMETER_HOST_IPV4_ADDRESS ipdiameter.ipv4_address The IPV4 address of the local node.	**"10.1.2.1"** char *
DIAMETER_HOST_IPV6_ADDRESS ipdiameter.ipv6_address The IPV6 address of the local node.	**"3ffe:300::5"** char *
DIAMETER_PEER_MAX_RECONN_ATTEMPT ipdiameter.maxreconnattempt Specifies the maximum number of peer re-connection attempts before dropping the call.	**"30"** char *
DIAMETER_REQ_MAX_RETX ipdiameter.maxreqretx Specifies the maximum number of retransmit requests to issue before declaring a failure for an expected response to a request.	**"3"** char *
DIAMETER_PEER_RECONN_INTERVAL ipdiameter.timeout.reconninterval Sets the interval for transport connection attempts to a peer with whom no active transport connection exists. The recommended value is 30 seconds. The value is in seconds.	**"30"** char *
DIAMETER_HOST_REALM ipdiameter.realm Realm of the Diameter node.	**"example.com"** char *
DIAMETER_REQ_RETX_INTERVAL ipdiameter.timeout.reqretxinterval Sets the interval for retransmit requests. The value is in seconds.	**"10"** char *
DIAMETER_SESSION_TIMEOUT ipdiameter.timeout.session Specifies the timeout before a session requires re-authentication. The value is in seconds.	**"30"** char *
DIAMETER_HOST_TCP_PORT ipdiameter.tcp_port The TCP listening port number of the local node.	**"3868"** char *
DIAMETER_VENDOR_ID ipdiameter.vendorid Specifies the IANA SMI Network Management Private Enterprise Codes value assigned to the vendor of the Diameter application.	**"0"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_WATCHDOG_TIMEOUT **ipdiameter.timeout.watchdog** Specifies the Diameter transport watchdog timeout interval. The value is in seconds.	**"30"** char *

3.7. Diameter Proxy Agent Configuration Parameters

The Diameter Proxy Agent Configuration Parameters on page 37 table lists the Diameter proxy agent configuration parameters and system variables.

Table 1. Diameter Proxy Agent Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_APPLICATION_LIST **ipdiameter.appl_list** Defines the list of application identifiers supported by the Diameter local node. The format is a comma-separated list of application identifiers. The options are Diameter NASREQ (1), Diameter Base Accounting (3), Diameter Credit Control (4), or Diameter EAP(5).	**"5"** char *
DIAMETER_HOST_DOMAIN **ipdiameter.domain** Diameter local node address family. Set to 2 for IPv4 or to 28 for IPv6.	**"2"** char *
DIAMETER_IDENTIFIER **ipdiameter.identifier** Diameter entity identity, which is the node's fully qualified domain name (FQDN). This identity string is used to get the node's address with **getaddrinfo( )**. The user must enter a valid and resolvable entry for this string.	**"apple.example. com"** char *
DIAMETER_HOST_IPV4_ADDRESS **ipdiameter.ipv4_address** The IPV4 address of the local node.	**"10.1.2.1"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_HOST_IPV6_ADDRESS **ipdiameter.ipv6_address** The IPV6 address of the local node.	**"3ffe:300::5"** char *
DIAMETER_PEER_MAX_RECONN_ATTEMPT **ipdiameter.maxreconnattempt** Specifies the maximum number of peer re-connection attempts before dropping the call.	**"30"** char *
DIAMETER_REQ_MAX_RETX **ipdiameter.maxreqretx** Specifies the maximum number of retransmit requests to issue before declaring a failure for an expected response to a request.	**"3"** char *
DIAMETER_PEER_RECONN_INTERVAL **ipdiameter.timeout.reconninterval** Sets the interval for transport connection attempts to a peer with whom no active transport connection exists. The recommended value is 30 seconds. The value is in seconds.	**"30"** char *
DIAMETER_HOST_REALM **ipdiameter.realm** Realm of the Diameter node.	**"example.com"** char *
DIAMETER_REQ_RETX_INTERVAL **ipdiameter.timeout.reqretxinterval** Sets the interval for retransmit requests. The value is in seconds.	**"10"** char *
DIAMETER_HOST_TCP_PORT **ipdiameter.tcp_port** The TCP listening port number of the local node.	**"3868"** char *

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_VENDOR_ID **ipdiameter.vendori** Specifies the IANA SMI Network Management Private Enterprise Codes value assigned to the vendor of the Diameter application.	**"0"** char *
DIAMETER_WATCHDOG_TIMEOUT **ipdiameter.timeout.watchdog** Specifies the Diameter transport watchdog timeout interval. The value is in seconds.	**"30"** char *

3.8. Configuring TLS

There are four possible ways to configure TLS:

- both the client and the proxy have TLS and neither has the secure key database. You can use the same VSB as the base for the client and proxy VIPs.
- the client has TLS and the secure key database and the proxy has TLS but does not have the secure key database. You must create one VSB for the client VIP and one VSB for the proxy VIP.
- the proxy has TLS and the secure key database and the client has TLS but does not have the secure key database. You must create one VSB for the client VIP and one VSB for the proxy VIP.
- both the client and the proxy have TLS and the secure key database. You can use the same VSB as the base for the client and proxy VIPs.

For information on using the secure key database, see the VxWorks 7 Cryptography Libraries Programmer's Guide and the .

VxWorks Source Build on page 39 In Workbench the **IPDIAMETER_SEC_KEY_DB** component is not visible until you enable **COMPONENT_IPDIAMETERC** or **COMPONENT_IPDIAMETERP**.
VxWorks Image Project on page 40 To use TLS the client or proxy agent VIP must also include the **INCLUDE_IPDIAMETER_TLS_CFG** component.
TLS Configuration Parameters on page 40
Using ROMGS for TLS on page 42 To use **romfs** for the purpose of making the certificates and the key available to TLS, do the following.
Configuring the Key Database Parameters on page 42

VxWorks Source Build

In Workbench the **IPDIAMETER_SEC_KEY_DB** component is not visible until you enable **COMPONENT_IPDIAMETERC** or **COMPONENT_IPDIAMETERP**.

The presence or absence of the **IPDIAMETER_SEC_KEY_DB** component in the VSB affects the presence or absence of a subset of **INCLUDE_IPDIAMETER_TLS_CFG** component parameters in the VIP. This is the reason why the same VSB can sometimes be used for both the client and the proxy and why sometimes the client and proxy require different VSBs.

If the **IPDIAMETER_SEC_KEY_DB** component is not enabled in the VSB and you include the **INCLUDE_IPDIAMETER_TLS_CFG** component in the VIP, you will see the **INCLUDE_IPDIAMETER_TLS_CFG** component as having the following parameters:

- **DIAMETER_TLS_ENABLED**
- **DIAMETER_TLS_ROOT_CERTIFICATE**
- **DIAMETER_TLS_CERTIFICATE**
- **DIAMETER_TLS_ROOT_DIR**
- **DIAMETER_TLS_KEY_FILE**
- **DIAMETER_TLS_KEY_PASSWORD**

If the **IPDIAMETER_SEC_KEY_DB** component is enabled and you include the **INCLUDE_IPDIAMETER_TLS_CFG** component in the VIP, you will see the **INCLUDE_IPDIAMETER_TLS_CFG** component as having the following parameters:

- **DIAMETER_TLS_ENABLED**
- **DIAMETER_TLS_ROOT_CERTIFICATE**
- **DIAMETER_TLS_CERTIFICATE**
- **DIAMETER_TLS_ROOT_DIR**
- **DIAMETER_TLS_KEY_ID**

See Source Build Configuration Options (VSB) on page 31 for instruction in how to add the Diameter client, proxy agent, and secure key database components to the VSB.

Parent topic: Configuring TLS on page 39

VxWorks Image Project

To use TLS the client or proxy agent VIP must also include the **INCLUDE_IPDIAMETER_TLS_CFG** component.

If **IPDIAMETER_SEC_KEY_DB** is enabled in the VSB, including **INCLUDE_IPDIAMETER_TLS_CFG** will automatically include **INCLUDE_IPCOM_USE_KEY_DB**.

To import the private key file into the secure key database from a remote host, include **INCLUDE_IPCOM_FS_NATIVE** in the VIP.

To import the private key file from a remote host using the **keydb** console command, include the **INCLUDE_IPCOM_KEY_DB_CMD** component in the VIP.

Parent topic: Configuring TLS on page 39

TLS Configuration Parameters

The TLS Configuration Parameters on page 40 table lists the TLS configuration parameters and their default values. Some of these parameters reference encryption certificate and key files. See Openssl Example on page 89 for instructions on how to generate sample certificates and keys for use in the testing of the TLS feature.

Table 1. TLS Configuration Parameters

Parameter Name, System Variable Name, and Description	Default and Type
DIAMETER_TLS_ENABLED **ipdiameter.tls** Specifies whether TLS is enabled or not on the local node. Set to **1** to enable or set to **0** to disable. If TLS is enabled, IPSSL and its requirements must be	**"0"** char *

Parameter Name, System Variable Name, and Description	Default and Type
built, the remaining Diameter TLS configuration parameters must be configured properly, and certificate files and private key files must be loaded.	
DIAMETER_TLS_ROOT_CERTIFICATE **ipdiameter.tls.cacertfile** Specifies the certificate of authorization (CA) root certificate file.	**"cacertfile.pem"** char *
DIAMETER_TLS_CERTIFICATE **ipdiameter.tls.certfile** Specifies the local node's certificate file.	**"certfile.pem"** char *
DIAMETER_TLS_ROOT_DIR **ipdiameter.tls.rootdir** Specifies the location of the certificate of authorization (CA) root directory.	**""** char *
DIAMETER_TLS_KEY_PASSWORD **ipdiameter.tls.keypasswd** Specifies the client key for the local node's private key file. This parameter is visible if **IPDIAMETER_SEC_KEY_DB** is not enabled in the VSB.	**"clientkey"** char *
DIAMETER_TLS_KEY_FILE **ipdiameter.tls.keyfile** Specifies the local node's private key file. This parameter is visible if **IPDIAMETER_SEC_KEY_DB** is not enabled in the VSB.	**"keyfile.pem"** char *
DIAMETER_TLS_KEY_ID **ipdiameter.tls.tlsPKeyId** This KeyId will be used to retrieve the private key file from secure key database. If **IPDIAMETER_SEC_KEY_DB** is enabled in the VSB then the user has to import the private key file into the secure key database using this keyid in order to be able to retrieve it at run time. Otherwise diameter with TLS security will not work.	**"diameterTlsPKeyId"** char *

Parent topic: Configuring TLS on page 39

Using ROMGS for TLS

To use **romfs** for the purpose of making the certificates and the key available to TLS, do the following.

1. 1. Add a ROMFS project to the VIP. See Using ROMFS to Store Certificates on page 92.
2. 2. Set DIAMETER_TLS_ENABLED to "1".
3. 3. If the VIP contains the secure key database components, you see the DIAMETER_TLS_ROOT_DIR, DIAMETER_TLS_ROOT_CERTIFICATE, DIAMETER_TLS_CERTIFICATE, and DIAMETER_TLS_KEY_ID parameters.
    1. Set DIAMETER_TLS_ROOT_DIR to "/romfs".
    2. Set DIAMETER_TLS_ROOT_CERTIFICATE to "/romfs/cacertfile.pem".
    3. Set DIAMETER_TLS_CERTIFICATE to "/romfs/certfile.pem".
    4. Leave the value of the DIAMETER_TLS_KEY_ID parameter as-is.
4. If the VIP does not contain the secure key database components, you see the DIAMETER_TLS_ROOT_DIR, DIAMETER_TLS_ROOT_CERTIFICATE, DIAMETER_TLS_CERTIFICATE, DIAMETER_TLS_KEY_FILE, and DIAMETER_TLS_KEY_PASSWORD parameters.
    1. Set DIAMETER_TLS_ROOT_DIR to "/romfs".
    2. Set DIAMETER_TLS_ROOT_CERTIFICATE to "/romfs/cacertfile.pem".
    3. Set DIAMETER_TLS_CERTIFICATE to "/romfs/certfile.pem".
    4. Set DIAMETER_TLS_KEY_FILE to "/romfs/keyfile.pem".
    5. Set DIAMETER_TLS_KEY_PASSWORD to "SomePassword".

Parent topic: Configuring TLS on page 39

Configuring the Key Database Parameters

For information on configuring the secure key database, see the VxWorks 7 Cryptography Libraries Programmer's Guide.

> Note: The **INCLUDE_IPCOM_USE_KEY_DB** component has one parameter, SEC_VAULT_KEY_ENCRYPTING_PW. Its default value is the empty string. It must be set to a random string of 40 characters or the VIP build fails.

Parent topic: Configuring TLS on page 39

4. Shell Commands
4.1. Introduction

You can use the Diameter client shell command (diameterc) to send an access request to a node, show configuration information, manage peer entries, manage sessions, test interfaces, and manage the client state.

You can use the Diameter proxy agent shell command (diameterp) to show configuration information, manage peer entries, manage policies, manage routes, test interfaces, and manage the proxy agent state.

You can use the keydb shell command to import the key file into the secure key database.

You can use the sysvar shell command to modify Diameter configuration parameters. The Diameter Client Configuration Parameters on page 35 table correlates Diameter client VIP configuration parameters to their respective system variables. The Diameter Proxy Agent Configuration Parameters on page 37 table correlates Diameter proxy agent VIP configuration parameters to their respective system variables. The TLS Configuration Parameters on page 40 table correlates TLS VIP configuration parameters to their respective system variables.

You can use the ipd shell command to view and change the state of the Diameter service (ipdiameterc).

You can use the syslog shell command to set the priority for the Diameter syslog debug messages (ipdiameter). For more information, see the VxWorks 7 Network Stack Programmer's Guide.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd


[vxWorks *} #
```

To return to the kernel shell, enter the following command:

```
# C

->
```

4.2. Diameter Client Shell Command

The Diameter client shell command (diameterc) is the easiest way to become familiar with the Diameter client.

To access the diameterc shell command mode, enter the following command at the kernel shell prompt:

```
-> cmd diameterc

diameterc>
```

The Diameter Client Shell Commands on page 43 table lists the Diameter client shell commands in alphabetical order and provides a brief description of each command.

Table 1. Diameter Client Shell Commands

Shell Command	Description
access -user *user* -pwd *pwd* -servip *ip*	Sends an access request to a node (test mode).
exit	Exits the Diameter shell.
help	Displays the shell commands and command modifiers.
nodeshow	Shows the node configuration.
openaccess -user *user* -pwd *pwd* -servip *ip* -realm *realm* -tcid *tcid* -rar *rarSupport*	Sets the test mode and sends an access request to an Open Diameter server. Set rarSupport to 1 when session re-authentication is supported, otherwise set to 0. The tcid modifier allows you to select the test mode. **The base1 mode enables single session testing.** **The base2 mode enables multi session testing.** **The eap mode enables EAP testing.**

Shell Command	Description
	The openaccess test command is only available when the **OPEN_DIAMETER_TEST** macro is configured for the project. For configuration information, see Enabling the Diameter Client Shell Test Command on page 33. To enable EAP testing, see Enabling the Testing of Diameter EAP on page 33.
peeradd -peerip *ip* -realm *realm* [-port *port*] [-tlsenabled *enabled*] [-peertype *peertype*]	Adds a peer entry.
peerdel -peerip *ip* [-port *port*]	Deletes a peer entry.
peermod -peerip *ip*	Changes a peer entry from failed to the closed state.
peershow	Shows peer entries.
sessiondel -sessionid *sessionid*	Deletes a session entry.
sessionshow	Shows session entries.
startc	Starts the Diameter client.
stopc	Stops the Diameter client.

Using the Diameter Client Shell Command on page 44
Testing a Peer Connection on page 49 The access shell command establishes a connection and a session to an active Diameter peer (node). Once connected, the peer automatically updates its peer table with the client coordinates.
Testing Server Connection on page 50 The openaccess shell command establishes a connection and a session to an Open Diameter server. The tcid modifier allows you to select the test mode.
Additional openaccess Modifiers on page 54 To specify a second Open Diameter server for the openaccess command, use the -servip2 modifier. For example:

Using the Diameter Client Shell Command

Getting Help

To display help for all commands and command modifiers, enter help. Command modifiers can be entered in any order.

```
diameterc> help
```

Viewing the Node Configuration

The following example shows how to display the node state and client parameter values. A **State** value of **opened and running** indicates that the Diameter client is ready.

```
diameterc> nodeshow
State:                  opened and running
Node type:              Client
```

```
Node Addr:             10.1.2.1
Diameter Id:           apple.example.com
Realm Id:              example.com
Vendor Id:             0
TCP listening port:    3868
Supported application: (5)Diameter EAP


Timers:
  Session timeout:                30
  Authorization lifetime:         360
  Auth Grace Period:              30
  Peer reconnection interval:     30
  Max reconnection attemps:       30
  Watchdog timeout:               30
  Request retx interval:          10
  Max request retx:               3

TLS:
  TLS enabled:                    Yes
  Node certficate file:           certfile.pem
  Node private key file:          keyfile.pem
  Node key file's key passwd:     clientkey
  CA root certificate file:       cacertfile.pem
  Certificate root directory:     /romfs
```

To configure node parameters from an application, you must convert the command shell functions to callable APIs. For more information, see Executing Shell Commands from an Application on page 70.

The **State** field value can be opened and running or opened and NOT running.

The **Node type** field value can be Unknown (0), Access Server (1), Acct Server (2), Full Server (3), Client (4), Relay (5), Redirect (6), Proxy (7), or Translate (8). The Unknown type indicates a dynamically discovered node.

The **Supported application** field values include Diameter NASREQ (1), Diameter Base Accounting (3), Diameter Credit Control (4), or Diameter EAP(5).

Managing Peers

Diameter peers are nodes that provide Diameter functionality in a network. Diameter nodes can be configured as one or more of the following:

- Client
- Relay agent
- Proxy agent
- Redirect agent
- Translation agent
- Server (access, accounting, or full)

Each node has a list of peers used for networking purposes. To view the list of peers, use the peershow command. You can add peers with the peeradd command and delete peers with the peerdel command. You can also change the peer state from failed to closed (unavailable) with the peermod command. A peer set to the closed state can be reinstated automatically when the node, through re-connection attempts, reasserts the connection with the targeted peer.

Use the -tlsenabled modifier to indicate that the peer is TLS-enabled. The default is TLS-disabled.

Displaying Diameter Peers

The following example shows how to display the active Diameter peers.

```
diameterc> peershow
Diameter Peers:            1
Diameter peer (162d774):
  Peer Addr:               172.16.150.246
  Peer Addr port:          7183
  Peer Node Type:          Full Server
  Peer Realm:              homedomain.com
  Peer Host name:          172.16.150.246
  Peer is dynamic:         No
  Peer tls enabled:        No
  Peer state:              Open
  Peer fd:                 27
  Peer TLS connection:     No
```

Adding and Deleting Diameter Peers

The following example shows how to add a peer and delete a peer. When a port modifier is not specified, the port number is set to 3868.

```
diameterc> peeradd -peerip 172.16.150.100 -realm homedomain.com -tlsenabled 0
peeradd -peerip 172.16.150.100 -realm homedomain.com -tlsenabled 0
Added Diameter peer entry
diameterc>

diameterc> peeradd -peerip 172.16.150.101 -realm homedomain.com -port 2020 -tlsenabled 0
peeradd -peerip 172.16.150.101 -realm homedomain.com -port 2020 -tlsenabled 0
Added Diameter peer entry
diameterc>

diameterc> peerdel -peerip 172.16.132.122
Removed Diameter peer entry

diameterc> peerdel -peerip 172.16.132.124 -port 2020
Removed Diameter peer entry
```

Changing the State of Diameter Peers

The following example shows how to change a peer state from failed to closed (unavailable).

```
diameterc> peershow
Diameter Peers:          1
Diameter peer (162d884):
  Peer Addr:             172.16.150.246
  Peer Addr port:        7183
  Peer Node Type:        Proxy
  Peer Realm:            homedomain.com
  Peer Host name:        172.16.150.245
  Peer is dynamic:       No
  Peer tls enabled:      No
  Peer state:            Failed
```

```
  Peer fd:              -1
  Peer TLS connection:   No



diameterc> peermod -peerip 172.16.150.246
Reset Diameter peer entry state to closed

diameterc> peershow
Diameter Peers:         1
Diameter peer (162d884):
  Peer Addr:            172.16.150.246
  Peer Addr port:       7183
  Peer Node Type:       Proxy
  Peer Realm:           homedomain.com
  Peer Host name:       172.16.150.245
  Peer is dynamic:      No
  Peer tls enabled:     No
  Peer state:           Closed
  Peer fd:              -1
  Peer TLS connection:  No
```

Managing Sessions

A Diameter session is a logical connection between two Diameter nodes and can cross multiple connections. A session is defined as the interaction between a Diameter client and a Diameter server over a given time frame. For more information on session identifiers, see RFC 3588.

Displaying Active Diameter Sessions

The following example shows how to display the active Diameter sessions.

```
diameterc> sessionshow
Diameter Sessions:         1
Diameter session (13b7da0):
  Session ID:              hac.homedomain.com.homedomain.net;1182441072;424728
  Session appl_id:         EAP
  Session is stateful:     No
  Session state:           Pending
  Next hop:                172.16.150.246
  Selected server index:   0
  Tried servers(bit map):  1
  Supplied server:         172.16.150.246
```

Terminating Active Diameter Sessions

The following example shows how to terminate a Diameter session. Use the sessionshow command to obtain the session identifier and the sessiondel command to delete the session. This command sends a Session Terminate Request (STR) to the active server, deletes any pending requests associated with the session, deletes the session, and sends a session-delete event to the application.

```
diameterc> sessionshow
Diameter Sessions:          1
Diameter session (13b7da0):
  Session ID:               hac.homedomain.com.homedomain.net;1182441072;424728
```

```
  Session appl_id:          EAP
  Session is stateful:      No
  Session state:            Pending
  Next hop:                 172.16.150.246
  Selected server index:    0
  Tried servers(bit map):   1
  Supplied server:          172.16.150.246

diameterc> sessiondel -sessionid hac.homedomain.com.homedomain.net;1182441072;424728

diameterc> sessionshow
Diameter Sessions: None
```

Stopping and Starting the Diameter Daemon

When you load an image containing Diameter into a target and start it, the Diameter daemon starts automatically. The following example shows how to stop and restart a Diameter daemon.

```
diameterc> stopc
Diameter client daemon is stopped.

diameterc> nodeshow
Diameter is not opened

diameterc> startc
Diameter client daemon is started.

diameterc> nodeshow
State:                 opened and running
Node type:             Client
Node Addr:             10.1.2.1
Diameter Id:           apple.example.com
Realm Id:              example.com
Vendor Id:             0
TCP listening port:    3868
Supported application: (5)Diameter EAP


Timers:
  Session timeout:             30
  Authorization lifetime:      360
  Auth Grace Period:           30
  Peer reconnection interval:  30
  Max reconnection attemps:    30
  Watchdog timeout:            30
  Request retx interval:       10
  Max request retx:            3

TLS:
  TLS enabled:                 No
  Node certficate file:        N/A
  Node private key file:       N/A
  Node key file's key passwd:  N/A
  CA root certificate file:    N/A
  Certificate root directory:  N/A
```

Parent topic: Diameter Client Shell Command on page 43

Testing a Peer Connection

The access shell command establishes a connection and a session to an active Diameter peer (node). Once connected, the peer automatically updates its peer table with the client coordinates.

The following example shows the results of a successful connection to a Diameter peer. The Diameter peer is at address **172.16.150.246** with the **homedomain.com** realm. The following commands are used in the example:

- The peeradd command adds the peer to the client peer table.
- The nodeshow command displays the state of the node. A **State** of opened and running indicates a node ready to communicate with peers.
- The peershow command displays the peers in the entry table. A **Peer state** of Closed indicates an unconnected peer. A **Peer Node Type** of Unknown indicates a dynamically discovered peer.

  The access command sends packets to the peer. A **Peer state** set to I-Open (initiate) indicates that the peer initiated the request and is connected. A **Peer state** set to R-Open (responder) indicates that the peer has accepted a connection request and is connected. A **Session state** set to Open indicates an active session. The displayed output from the access command of this example indicates a successful connection to a peer.

```
diameterc> nodeshow
State:                  opened and running
Node type:              Client
Node Addr:              10.1.1.2
Diameter Id:            test.homedomain.com
Realm Id:               homedomain.com
Vendor Id:              0
TCP listening port:     3868
Supported application: (5)Diameter EAP

Timers:
  Session timeout:            30
  Authorization lifetime:     360
  Auth Grace Period:          30
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A

diameterc> peeradd -servip 172.16.150.246 -realm homedomain.com
diameterc> peershow
Diameter Peers:               1
Diameter peer (1631010):
  Peer Addr:                  172.16.150.246
  Peer Addr port:             4837
  Peer Node Type:             Unknown
```

```
  Peer Realm:                 homedomain.com
  Peer Host name:             172.16.150.246
  Peer is dynamic:            Yes         (Peer is dynamically discovered)
  Peer tls enabled:           No
  Peer state:                 Closed      (Peer is not connected)
  Peer fd:                    27
  Peer TLS connection:        No
  Peer supported application: (1)Diameter NASREQ
  Peer supported application: (2)Diameter Mobile-IP
  Peer supported application: (3)Diameter Base Accounting
  Peer supported application: (4)Diameter Credit Control

diameterc> access -servip 172.16.150.246 -user suni@homedomain.com -pwd request -realm homedomain.
com
access -servip 172.16.150.246 -user suni@homedomain.com -pwd request
-realm homedomain.com
...
diameterc> peershow
Diameter Peers:                1
Diameter peer (1631010):
  Peer Addr:                   172.16.150.246
  Peer Addr port:              4837
  Peer Node Type:              Unknown
  Peer Realm:                  homedomain.com
  Peer Host name:              172.16.150.246
  Peer is dynamic:             Yes
  Peer tls enabled:            No
  Peer state:                  R-Open       (Peer is connected)
  Peer fd:                     27
  Peer TLS connection:         No
  Peer supported application: (1)Diameter NASREQ
  Peer supported application: (2)Diameter Mobile-IP
  Peer supported application: (3)Diameter Base Accounting
  Peer supported application: (4)Diameter Credit Control

diameterc> sessionshow
sessionshow
Diameter Sessions
Diameter session (1627cec):
  Session ID:              test.homedomain.com;3;0;0
  Session appl_id:         Common Messages
  Session is stateful:     No
  Session state:           Pending      (Session is still in the pending state)
  Next hop:                172.16.150.246
  Selected server index:   0
  Tried servers(bit map):  1
  Supplied server:         172.16.150.246
```

From the peer, request a peershow and observe that a dynamic peer was added to the peer list.

Parent topic: Diameter Client Shell Command on page 43

Testing Server Connection

The openaccess shell command establishes a connection and a session to an Open Diameter server. The tcid modifier allows you to select the test mode.

- The base1 mode enables single session testing.
- The base2 mode enables multi session testing.
- The eap mode enables EAP testing.

Before you can use the openaccess command, you must define the **OPEN_DIAMETER_TEST** macro in the project.

The following example shows example output for a successful connection to an Open Diameter server. The Open Diameter server is at address **172.16.150.246** with the **homedomain.com** realm. The following commands are used in the example:

- The nodeshow command displays the state of your node. A **State** of opened and running indicates a node ready to communicate with peers.
- The peeradd command adds an Open Diameter peer to the entry table.
- The peershow command displays the peers in the entry table. A **Peer state** of Closed indicates an unconnected peer.
- The openaccess command sets the test mode and sends packets to the peer. A **Peer state** set to I-Open (initiate) indicates that the peer initiated the request and is connected. A **Peer state** set to R-Open (responder) indicates that the peer has accepted a connection request and is connected. A **Session state** set to Open indicates an active session. The displayed output from the event callback (**ipdiameter_example_event_callback: rx event access_accept**) in this example indicates a successful connection to an Open Diameter server.

```
diameterc> nodeshow
nodeshow
State:                 opened and running
Node type:             Client
Node Addr:             10.1.1.2
Diameter Id:           test.homedomain.com
Realm Id:              homedomain.com
Vendor Id:             0
TCP listening port:    3868
Supported application: (5)Diameter EAP


Timers:
  Session timeout:            30
  Authorization lifetime:     360
  Auth Grace Period:          30
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A

diameterc> peeradd -peerip 172.16.150.246 -realm homedomain.com
peeradd -peerip 172.16.150.246 -realm homedomain.com
added Diameter peer entry

diameterc> peershow
peershow
Diameter Peers
```

```
Diameter peer (16279b8):
  Peer Addr:            172.16.150.246
  Peer Addr port:       3868
  Peer Node Type:       Full Server
  Peer Realm:           homedomain.com
  Peer Host name:       172.16.150.246
  Peer is dynamic:      No
  Peer tls enabled:     No
  Peer state:           Closed
  Peer fd:              -1
  Peer TLS connection:  No

diameterc> openaccess -servip 172.16.150.246 -user suni@homedomain.com -pwd request -realm homedom
ain.com -tcid base1 -rar 1
openaccess -servip 172.16.150.246 -user suni@homedomain.com -pwd request -realm homedomain.com -tc
id base1 -rar 1

ipdiameter_pkt_send -> tx DIAMETER packet [code=257, flags=80, length=132, appl_id=0, hbh_id=13936
29294, e2e_id=1393629294]
diameterc> ipdiameter_recv, rcv pkt on fd 16, from 172.16.150.246
ipdiameter_read <- rcv DIAMETER packet, [code=257, flags=0, length=300, appl_id=0, hbh_id=13936292
94, e2e_id=1393629294], from 172.16.150.246
ipdiameter_pkt_send -> tx DIAMETER packet [code=260, flags=80, length=388, appl_id=2, hbh_id=28490
33555, e2e_id=2849033555]
ipdiameter_recv, rcv pkt on fd 16, from 172.16.150.246
ipdiameter_read <- rcv DIAMETER packet, [code=260, flags=0, length=256, appl_id=2, hbh_id=28490335
55, e2e_id=2849033555], from 172.16.150.246
ipdiameter_example_event_callback: rx event access_accept

Diameter request (162b064):
  Request cookie:        17efe18
  Request unique:        2849033555
  Request session ID:    hac.homedomain.com.homedomain.net;1182441072;424728
  Request appl ID:       2
  Request pwd_type:      1
  Request pwd:           request
  Request chap_ident:    1
  Selected server index: 0
  Request server_addr:   172.16.150.246
  Request server_type:   Full Server

Diameter Peers
Diameter peer (16279b8):
  Peer Addr:              172.16.150.246
  Peer Addr port:         3868
  Peer Node Type:         Full Server
  Peer Realm:             homedomain.com
  Peer Host name:         172.16.150.246
  Peer is dynamic:        No
  Peer tls enabled:       No
  Peer state:             I-Open       (Showing the peer is connected)
  Peer fd:                16
  Peer TLS connection:    No

Diameter Sessions
(Showing a session with ID
hac.homedomain.com.homedomain.net;1182441072;424728 is created and opened)
```

```
Diameter session (161528c):
  Session ID:          hac.homedomain.com.homedomain.net;1182441072;424728
  Session appl_id:     EAP
  Session is stateful: Yes
  Session state:       Open
  Next hop:               172.16.150.246
  Selected server index:  0
  Tried servers(bit map): 1
  Supplied server:        172.16.150.246

ipdiameter_pkt_send -> tx DIAMETER packet [code=260, flags=80, length=388,
appl_id=2, hbh_id=4000477524, e2e_id=4000477524]
ipdiameter_recv, rcv pkt on fd 16, from 172.16.150.246
ipdiameter_read <- rcv DIAMETER packet, [code=260, flags=0, length=256,
appl_id=2, hbh_id=4000477524, e2e_id=4000477524], from 172.16.150.246

ipdiameter_example_event_callback: rx event access_accept
Diameter request (1631064):
  Request cookie:         17efe18
  Request unique:         4000477524
  Request session ID:     hac.homedomain.com.homedomain.net;1182441072;424729
  Request appl ID:        2
  Request pwd_type:       1
  Request pwd:            request
  Request chap_ident:     1
  Selected server index:  0
  Request server_addr:    172.16.150.246
  Request server_type:    Full Server
Diameter Sessions

ipdiameter_example_event_callback: rx event session_del
ipdiameter_example_event_callback: will send out STR for session
hac.homedomain.com.homedomain.net;1182441072;424728

(Terminating a session - hac.homedomain.com.homedomain.net;1182441072;424728)
ipdiameter_pkt_send -> tx DIAMETER packet [code=275, flags=80, length=180,
appl_id=0, hbh_id=1448245423, e2e_id=1448245423]
ipdiameter_example_event_callback: rx event session_discon
ipdiameter_recv, rcv pkt on fd 16, from 172.16.150.246
ipdiameter_read <- rcv DIAMETER packet, [code=275, flags=0, length=188,
appl_id=0, hbh_id=1448245423, e2e_id=1448245423], from 172.16.150.246
ipdiameter_example_event_callback: rx event session_del
ipdiameter_example_event_callback: will send out DPR

(Disconnecting the peer)
ipdiameter_pkt_send -> tx DIAMETER packet [code=282, flags=80, length=84,
appl_id=0, hbh_id=1467576775, e2e_id=1467576775]
ipdiameter_recv, rcv pkt on fd 16, from 172.16.150.246
ipdiameter_read <- rcv DIAMETER packet, [code=282, flags=0, length=88,
appl_id=0, hbh_id=1467576775, e2e_id=1467576775], from 172.16.150.246

diameterc> peershow

peershow
Diameter Peers
Diameter peer (16279b8):
```

```
  Peer Addr:           172.16.150.246
  Peer Addr port:      3868
  Peer Node Type:      Full Server
  Peer Realm:          homedomain.com
  Peer Host name:      172.16.150.246
  Peer is dynamic:     No
  Peer tls enabled:    No
  Peer state:          Closed      (Peer disconnected)

  Peer fd:             -1
  Peer TLS connection: No

diameterc> sessionshow

Diameter Sessions: None      (All sessions are deleted)
diameterc>
```

Parent topic: Diameter Client Shell Command on page 43

Additional openaccess Modifiers

To specify a second Open Diameter server for the openaccess command, use the -servip2 modifier. For example:

```
diameterc> openaccess -servip 172.16.150.246 -user suni@homedomain.com -pwd request -realm homedom
ain.com -servip2 172.16.150.256 -tcid eap -rar 1
```

To specify the password type, use the -pwdtype modifier (1 = PAP and 2 = CHAP). The default password type is PAP. To specify the server type, use the -servtype modifier (1 = access, 2 = accounting, and 3 = full). The default server type is full. For example:

```
diameterc> openaccess -servip 172.16.150.246 -user suni@homedomain.com -pwd request -realm homedom
ain.com -pwdtype 2 -servtype full -tcid eap -rar 1
```

Parent topic: Diameter Client Shell Command on page 43

4.3. Diameter Proxy Agent Shell Command

The Diameter proxy agent shell command (diameterp) is the easiest way to become familiar with the Diameter proxy agent.

To access the diameterp shell command mode, enter the following command at the kernel shell prompt:

```
-> cmd diameterp
diameterp>
```

To return to the kernel shell, enter the following command:

```
diameterp> exit
->
```

Diameter Proxy Agent Shell Commands on page 55 lists the Diameter proxy agent shell commands in alphabetical order and provides a brief description of each command.

Table 1. Diameter Proxy Agent Shell Commands

Shell Command	Description
exit	Exits the Diameter shell.
help	Displays the shell commands.
nodeshow	Shows node configuration.
peeradd -peerip *ip* -realm *realm* [-port *port*] [-tlsenabled *enabled*] [-peertype *peertype*]	Adds a peer entry.
peerdel -peerip *ip* [-port *port*]	Deletes a peer entry.
peermod -peerip *ip*	Modifies the state of a peer entry.
peershow	Shows peer entries.
policyadd [-username *username*] [-hostid *hostname*]	Adds a diameter policy entry (proxy agent).
policydel [-username *username*] [-hostid *hostname*]	Deletes a diameter policy entry (proxy agent).
policyshow	Shows Diameter policy entries (proxy agent).
routeadd -realm *realmname* -application *application_id* -vendor *vendor_id* -nexthop *nexthopname*	Adds a diameter route entry.
routedel -realm *realmname* -application *application_id* -vendor *vendor_id* [-nexthop *nexthopname*]	Deletes a diameter route entry.
routeshow	Shows Diameter route entries.
startp	Starts the Diameter agent.
stopp	Stops the Diameter agent.
testrar -clientip *clientIpAddress* -servrealm *serverRealm* -sessionid *sessionId*	Issues a session reauthentication request from a Diameter proxy to a Diameter client.

Getting Help on page 56 To display shell help for the diameterp shell command, enter help. Command modifiers can be entered in any order.

Viewing the Node Configuration on page 56 The following example shows how to display the node state and parameter values. A **State** value of opened and running indicates that the Diameter client is ready.

Managing Peers on page 57 Diameter peers are nodes providing Diameter functionality in a network. Diameter nodes can be configured as one or more of the following:

Managing Policies on page 58 The policy enforcement list consists of a set of unauthorized usernames or host identifiers to be used by the proxy agent. If a packet is associated with an item in the policy enforcement list, the packet is discarded and a reply message with a permanent failure code of **DIAMETER_AUTHORIZATION_REJECTED** is forwarded to the downstream peer. Configured user names and host IDs must follow the NAI format as defined in RFC 4282.

Managing Routes on page 60 The Diameter agent searches the peer and/or routing tables to find the next hop. The agent first searches the peer table to find a peer whose destination ID matches the destination host AVP carried in the packet. If a match is

found, the specified peer is the next hop for the packet. Otherwise, the routing table is searched. A single routing entry is created for a realm, application ID, and vendor ID. Each routing entry contains a list of next hops for the packet.
Managing the Diameter Daemon on page 61 When you load an image containing Diameter into a target and start it, the Diameter daemon starts automatically. You can stop the daemon and restart the daemon from the Diameter shell.
Testing Session Re-authentication on page 62 You can test the session re-authentication process for a Diameter client. For this example, the re-authentication request is generated from a Diameter proxy with the use of the testrar shell command. After initial session authentication from the Diameter server, you issue a testrar command from a proxy node to request re-authentication.
Importing a Key File for a TLS Connection on page 63

Getting Help

To display shell help for the diameterp shell command, enter help. Command modifiers can be entered in any order.

```
diameterp> help
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Viewing the Node Configuration

The following example shows how to display the node state and parameter values. A **State** value of opened and running indicates that the Diameter client is ready.

```
diameterp> nodeshow
State:                  opened and running
Node type:              Proxy
Node Addr:              10.1.1.2
Diameter Id:            proxy1.homedomain.com
Realm Id:               homedomain.com
Vendor Id:              0
TCP listening port:     3868
Supported application:(5)Diameter EAP

Timers:
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A
```

The **State** field value can be **opened and running** or **opened and NOT running**.

The **Node type** field value can be Unknown (0), Access Server (1), Acct Server (2), Full Server (3), Client (4), Relay (5), Redirect (6), Proxy (7), or Translate (8). The **Unknown** type indicates a dynamically discovered node.

The **Supported application** field values include Diameter NASREQ (1), Diameter Base Accounting (3), Diameter Credit Control (4), or Diameter EAP (5).

Parent topic: Diameter Proxy Agent Shell Command on page 54

Managing Peers

Diameter peers are nodes providing Diameter functionality in a network. Diameter nodes can be configured as one or more of the following:

- Client
- Relay agent
- Proxy agent
- Redirect agent
- Translation agent
- Server (access, accounting, or full)

Each node has a list of peers used for networking purposes. Use the peershow command to view the list of peers. You can add peers with the peeradd command and delete peers with the peerdel command. You can also change the peer state from failed to closed (unavailable) with the peermod command. A peer set to the closed state can be reinstated automatically when the node, through re-connection attempts, reasserts the connection with the targeted peer.

Use the -tlsenabled modifier to indicate the TLS-state of the peer (1 for enabled or 0 for disabled). The default is TLS-disabled.

Displaying Diameter Peers

The following example shows how to display the active Diameter peers

```
diameterp> peershow
peershow
Diameter Peers
Diameter peer (162d774):
  Peer Addr:           172.16.150.246
  Peer Addr port:      7183
  Peer Node Type:      Full Server
  Peer Realm:          homedomain.com
  Peer Host name:      172.16.150.246
  Peer is dynamic:     No
  Peer tls enabled:    No
  Peer state:          Open
  Peer fd:             27
  Peer TLS connection: No
```

Adding and Deleting Diameter Peers

The following example shows how to add a peer. When a port modifier is not specified, the port number is set to 3868.

```
diameterp> peeradd -peerip 172.16.150.100 -realm homedomain.com -tlsenabled 0
peeradd -peerip 172.16.150.100 -realm homedomain.com -tlsenabled 0
added Diameter peer entry
diameterp>

diameterp> peeradd -peerip 172.16.150.101 -realm homedomain.com -port 2020
peeradd -peerip 172.16.150.101 -realm homedomain.com -port 2020
```

```
added Diameter peer entry
diameterp>
```

The following example shows how to delete a peer. When a port modifier is not specified, port number 3868 is used.

```
diameterp> peerdel -peerip 172.16.132.122
removed Diameter peer entry

diameterp> peerdel -peerip 172.16.132.124 -port 2020
removed Diameter peer entry
```

Changing the State of Diameter Peers

The following example shows how to change a peer state from failed to closed (unavailable).

```
diameterp> peershow
peershow
Diameter Peers
Diameter peer (162d884):
  Peer Addr:           172.16.150.246
  Peer Addr port:      7183
  Peer Node Type:      Proxy
  Peer Realm:          homedomain.com
  Peer Host name:      172.16.150.245
  Peer is dynamic:     No
  Peer tls enabled:    No
  Peer state:          Failed
  Peer fd:            -1
  Peer TLS connection: No

diameterp> peermod -peerip 172.16.150.246
Reset Diameter peer entry state to closed

diameterp> peershow
peershow
Diameter Peers
Diameter peer (162d884):
  Peer Addr:           172.16.150.246
  Peer Addr port:      7183
  Peer Node Type:      Proxy
  Peer Realm:          homedomain.com
  Peer Host name:      172.16.150.245
  Peer is dynamic:     No
  Peer tls enabled:    No
  Peer state:          Closed
  Peer fd:             -1
  Peer TLS connection: No
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Managing Policies

The policy enforcement list consists of a set of unauthorized usernames or host identifiers to be used by the proxy agent. If a packet is associated with an item in the policy enforcement list, the packet is discarded and a reply message with a permanent failure code

of **DIAMETER_AUTHORIZATION_REJECTED** is forwarded to the downstream peer. Configured user names and host IDs must follow the NAI format as defined in RFC 4282.

Displaying Diameter Policies

The following example shows how to display the Diameter policies.

```
diameterp> policyshow

Diameter proxy local policies:2
Diameter policy (8a6e10):
  User Name: myname.homedomain.com

Diameter policy (8a6f40):
  Host Name: homedomain.com
```

Adding a Diameter Policy Entry - Single User

The following example shows how to add a Diameter policy entry for a single user.

```
diameterp> policyadd -username user.isp.net
Added a policy entry

diameterp> policyshow
Diameter proxy local policies: 1
Diameter policy (8a6e10):
  User Name: user@isp.net
```

Adding a Diameter Policy Entry - Entire Host

The following example shows how to add a Diameter policy entry for a host (realm). The policyadd command is used to add a new policy entry. Use the policyshow command to view the added policy entry.

```
diameterp> policyadd -hostid ispl.net
Added a policy entry

diameterp> policyshow
Diameter proxy local policies: 2
Diameter policy (8a6e10):
  User Name: user@isp.net

Diameter policy (8a6f40):
  Host Name: isp1.net
```

The following example shows how to delete a Diameter policy entry. Use the policyshow command to display the policy entries and use the policydel command to delete an entry from the policy enforcement list.

```
diameterp> policyshow
Diameter proxy local policies: 1
Diameter policy (8a6e10):
  User Name: user@isp.net

diameterp> policydel -username user@isp.net
```

```
Deleted a policy entry

diameterp> policyshow
Diameter proxy local policies: None
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Managing Routes

The Diameter agent searches the peer and/or routing tables to find the next hop. The agent first searches the peer table to find a peer whose destination ID matches the destination host AVP carried in the packet. If a match is found, the specified peer is the next hop for the packet. Otherwise, the routing table is searched. A single routing entry is created for a realm, application ID, and vendor ID. Each routing entry contains a list of next hops for the packet.

If a routing table entry is found, the packet is forwarded to the peer. If a connection with a peer is not established, the packet is added to the transmission queue. Once the connection has been successfully established, the packet is forwarded to the next hop.

The routing table may include a default entry that is used to forward packets if no matching routing table entry is found. The routing table accepts a single default entry and one next hop. If a route is not found, a protocol error message result code of **DIAMETER_UNABLE_TO_DELIVER** is generated and forwarded to the downstream peer.

Displaying Routing Table Entries

The following example shows how to display routing table entries.

```
diameterp> routeshow
Diameter routes:                    1
Diameter route (1877880):
  realm:                            isp.net
  route-type:                       static
  route-action-type:                proxy
  servers attached to this route :  1
  server:                           proxy2.isp.net
```

Deleting a Default Routing Table Entry

The following example shows how to delete a default routing table entry.

```
diameterp> routedel -realm * -nexthop server.isp.net
Diameter route entry is deleted
```

Creating a Routing Table Entry with Two Next Hops

The following example shows how to create a routing table with two next hops (server1 and server2). The order in which the routing table entries are entered in the table defines the sequence for selecting routes. In this case, the **server1.isp.net** route is selected first. If unable to route the packet, the **server2.isp.net** route is selected. This rule does not apply to the default route entry. The default route entry is always the last route in the sequence.

```
diameterp> routeadd -realm isp.net -application 2 -vendor 199
-nexthop server1.isp.net
Added Diameter route entry
```

```
diameterp> routeadd -realm isp.net -application 2 -vendor 199
-nexthop server2.isp.net
Added Diameter route entry

diameterp> routeshow
Diameter routes:
Diameter route (967d48):
realm:                          isp.net
vendor-id:                      2
application-id:                 999
route-type:                     static
route-action-type:              proxy
servers attached to this route : 1
server:                         server1.isp.net
server:                         server2.isp.net
```

Deleting a Next Hop from a Routing Table Entry

The following example shows how to delete the server1.isp.net next hop from the routing table entry.

```
diameterp> routedel -realm isp.net -application 2 -vendor 999
-nexthop server1.isp.net
Diameter route entry is deleted

diameterp> routeshow
Diameter routes:                  1
Diameter route (967d48):
  realm:                          isp.net
  vendor-id:                      2
  application-id:                 999
  route-type:                     static
  route-action-type:              proxy
  servers attached to this route : 1
  server:                         server2.isp.net
```

Deleting a Table Entry and all Next Hops for a Given Realm, Application, and Vendor

The following example shows how to delete a table entry and all next hops for a given realm, application, and vendor. Once deleted, all packets from the specified realm, application, and vendor are routed to the default routing table entry, if specified.

```
diameterp> routedel -realm isp.net -application 2 -vendor 999
Diameter route entry is deleted

diameterp> routeshow
Diameter routes: None
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Managing the Diameter Daemon

When you load an image containing Diameter into a target and start it, the Diameter daemon starts automatically. You can stop the daemon and restart the daemon from the Diameter shell.

Stopping and Restarting the Diameter Daemon

The following example shows how to stop and restart the Diameter daemon.

```
diameterp> stopp
Diameter proxy agent is stopped

diameterp> nodeshow
nodeshow: None
Diameter is not opened

diameterp> startp
Diameter proxy agent is started

diameterp> nodeshow
State:                  opened and running
Node type:              Proxy
Node Addr:              10.1.1.2
Diameter Id:            proxy1.homedomain.com
Realm Id:               homedomain.com
Vendor Id:              0
TCP listening port:     3868
Supported application:  (5)Diameter EAP

Timers:
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Testing Session Re-authentication

You can test the session re-authentication process for a Diameter client. For this example, the re-authentication request is generated from a Diameter proxy with the use of the testrar shell command. After initial session authentication from the Diameter server, you issue a testrar command from a proxy node to request re-authentication.

Ensure that the client has re-authentication support enabled (-rar modifier). For configuration information, see the Diameter Client Shell Commands on page 43 table (openaccess shell command).

Testing Session re-Authentication

To test session re-authentication for a client at 192.168.100.10 with a server realm of **server@serverrealm.com** and a session identifier of 22, proceed as follows from the proxy command shell:

```
diameterc> testrar -clientip 192.168.100.10 -servrealm server@serverrealmrealm.com -sessionid 22
Issued RAR on session 22 to client 192.168.100.10
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

Importing a Key File for a TLS Connection

To configure TLS at build time, see Configuring TLS.

If you did not include the secure key database feature at build time, you do not have to do anything at load time. The client daemon starts automatically and loads the certificates and key file required for TLS operation.

If you did include the secure key database feature at build time, you must, after bootup has completed, import the key file into the secure key database and then start the client explicitly. To do so, your VIP must include the **INCLUDE_IPCOM_KEY_DB_CMD** component. When you import the key file from an external location, you will be prompted for a password if the private key is encrypted with a pass phrase. As the administrator, you know the pass phrase and no one can login to the node and obtain it.

The sequence for importing the key file follows. *URL* and *passphrase* are placeholders. The *URL* string has the form: *FQDN* **:** *path* where *FQDN* is a fully qualified domain name, for example, **tech.writer@windriver.com** and *path* is a forward-slash path, for example **/UserAccounts/UserAccount/openssl/keys/keyfile.pem**.

```
# keydb import key diameterTlsPKeyId URL
Enter PEM pass phrase:passphrase
keydb list key
diameterTlsPKeyId

# diameterc
diameterc> startc
startc
Diameter client daemon is started.
```

For more information about the keydb command see the VxWorks 7 Cryptography Libraries Programmer's Guide.

When the peeradd command is run, it results in the TLS protocol being run as part of the connection setup.

You can see the status of the connection by running the peershow command. For example:

```
# peershow
Diameter Peers: 1
Diameter peer (1053b318):
 Peer Addr: 192.168.10.3
 Peer Addr port: 3868
 Peer Node Type: Proxy
 Peer Realm: proxy.com
 Peer Host name: 192.168.10.3
 Peer is dynamic: No
 Peer tls enabled: Yes
 Peer state: Wait-I-CEA
 Peer fd:  8
 Peer TLS connection: Established
```

Parent topic: Diameter Proxy Agent Shell Command on page 54

5. API Functions

5.1. Introduction

Once you load and start the Diameter client or agent, you can access the Diameter API functions from the host shell or from another application.

Diameter functions called by a network task execute in the context by which they are called. The functions are protected from race conditions using multiple internal Diameter mutexes.

Because Diameter uses global data structures, there can only be one copy of Diameter in a single module. Diameter can handle multiple servers and multiple session-transactions.

The Diameter constants and default parameter values are defined in **ipdiameter_constant.h**. The header file includes the following items:

- Basic Diameter constants and default parameter values
- Diameter header command codes
- Diameter application identifiers
- Diameter AVP codes
- RADIUS attribute tags
- Diameter return codes
- Diameter error values
- Diameter event callback events

Parameter values specified in **ipdiameter_config.c**, Workbench, and **wrtool** have precedence over the default parameter values defined in **ipdiameter_constant.h**.

5.2. Diameter API Functions

The Diameter API Routines on page 64 table lists the API functions available to Diameter applications and gives a brief description of each function. For more information, see the reference entries for Diameter (ipdiameter). The API functions are defined in the source files located in **installDir/vxworks-7/pkgs_v2/net/ipnet/aaa-***x.x.x.x***/diameter/src**.

For implementation details, see Applications on page 66.

Table 1. Diameter API Routines

Routine	Description
ipdiameter_access_request( )	Handle a user request.
ipdiameter_answer_pkt_free( )	Free a DIAMETER answer packet.
ipdiameter_answer_pkt_malloc( )	Allocate a DIAMETER answer packet.
ipdiameter_avp_add_array( )	Add a byte array AVP to packet.
ipdiameter_avp_add_host_addr( )	Add the Host-IP-Address AVP to packet.
ipdiameter_avp_add_ip_addr( )	Add an IP-Address AVP to packet.

Routine	Description
ipdiameter_avp_add_NAS_identifier( )	Add the NAS-Identifier AVP to packet.
ipdiameter_avp_add_origin_host( )	Add the Origin-Host AVP to packet.
ipdiameter_avp_add_origin_realm( )	Add the Origin-Realm AVP to packet.
ipdiameter_avp_add_u32( )	Add an unsigned 32-bit AVP to packet.
ipdiameter_avp_change_u32( )	Modify an unsigned 32-bit AVP data in a packet.
ipdiameter_avp_get_next( )	Get next AVP after the specified AVP.
ipdiameter_avp_get_session_id( )	Get the Session-Id AVP from a diameter packet.
ipdiameter_cmd_diameterc( )	Diameter client shell commands.
ipdiameter_cmd_diameterp( )	Diameter proxy agent shell commands.
ipdiameter_deregister_access_function( )	Deregister a user callback function.
ipdiameter_eap_access_request( )	Handle a user auth request.
ipdiameter_eap_free_DER( )	Free a Diameter EAP Request structure.
ipdiameter_get_avp( )	Find the specified AVP from a diameter packet.
ipdiameter_get_hdr_code( )	Get the command code in a DIAMETER packet.
ipdiameter_get_hdr_length( )	Get the length value in a DIAMETER packet.
ipdiameter_get_session_id( )	Get an unique session ID.
ipdiameter_get_unique_id( )	Get an unique random 32 bit ID.
ipdiameter_grouped_avp_add_array( )	Add a byte array AVP to a grouped AVP.
ipdiameter_grouped_avp_add_u32( )	Add an unsigned 32-bit AVP to a grouped AVP.
ipdiameter_grouped_avp_get_avp( )	Find the specified AVP from a grouped AVP.
ipdiameter_grouped_avp_get_next( )	Get next AVP after the specified AVP from a grouped AVP.
ipdiameter_peer_pkt_send( )	Send a Diameter package out to a peer.
ipdiameter_register_access_function( )	Register a user callback function.
ipdiameter_request_pkt_free( )	Free a DIAMETER request packet.

Routine	Description
ipdiameter_request_pkt_malloc( )	Allocate a DIAMETER request packet.
ipdiameter_session_delete( )	Delete a session by the session ID.
ipdiameter_set_hdr_code( )	Set the command code in a DIAMETER packet.
ipdiameter_set_hdr_length( )	Set the length in a DIAMETER packet.

6. Applications
6.1. Developing a Diameter Application

You can use the ipdiameter_cmd_opendiameter_access( ) test function and the ipdiameter_example_event_callback( ) callback function as a baseline for your application. The **ipdiameter_cmd_diameterc.c** source file includes both functions. The test function interfaces a client to a Diameter server and simulates three diameter sessions. The **callback** function notifies the test function of server reply packets and failure events. For example, when incoming Diameter packets are received on a TCP socket, a Diameter task can transfer the packets to your application. A callback function within your application can process the packets locally or call an IPC function to transfer the processing to another task.

For a description of the ipdiameter_cmd_opendiameter_access( ) test function, see Open Diameter Test Routine Overview on page 66. For information on the event callback structure, see Event Callback on page 67.

The openaccess client shell command detailed in Diameter Client Shell Command on page 43 makes use of the ipdiameter_cmd_opendiameter_access( ) and ipdiameter_example_event_callback( ) functions to execute client-to-server tests. For implementation details, see Testing a Server Connection on page 50.

Open Diameter Test Routine Overview on page 66
Event Callback on page 67
Diameter EAP Access Request on page 68 The ipdiameter_eap_access_request( ) function implements the Diameter EAP authentication request and performs the following functions:
Diameter EAP Callback on page 69 The EAP callback function is invoked when the Diameter EAP receives an answer from the Diameter client. This callback function parses the callback events, records the Diameter answer result, and invokes the registered EAP callback function to reply to the EAP authenticator.

Open Diameter Test Routine Overview

The ipdiameter_cmd_opendiameter_access( ) function implements the openaccess shell command functionality described in Testing a Server Connection on page 50. You can use this function as a template for any application that must interact with an Open Diameter server. The function is located in **ipdiameter_cmd_diameterc.c**. The API functions are described in the API reference entries.

Single Session and Multi-Session

The ipdiameter_cmd_opendiameter_access( ) function calls ipdiameter_cmd_alloc_access( ) to manage AVPs for the single session and multi-session. The ipdiameter_cmd_alloc_access( ) function calls the following API functions:

Single Session and Multi-Session

The ipdiameter_cmd_opendiameter_access( ) function calls ipdiameter_cmd_alloc_access( ) to manage AVPs for the single session and multi-session. The ipdiameter_cmd_alloc_access( ) function calls the following API functions:

- ipdiameter_set_hdr_code( ): Sets the command code to AA-Mobile-Node-Request.
- ipdiameter_set_hdr_code( ): Sets the command code to Open Diameter Test-Auth-Request.
- ipdiameter_example_event_callback( ): Sets the Diameter event callback.
- ipdiameter_avp_add_array( ): Adds the AVPs.
- ipdiameter_avp_add_origin_host( ): Adds the Origin-Host AVP.
- ipdiameter_avp_add_origin_realm( ): Adds the Origin-Realm AVP.
- ipdiameter_avp_add_array( ): Adds the User-Name AVP.
- ipdiameter_avp_add_array( ): Adds the IP-Diameter-CHAP-Password.
- ipdiameter_avp_add_u32( ): Adds the Auth-Application-ID AVP.
- ipdiameter_avp_add_u32( ): Adds the Re-Auth-Request-Type AVP.

When you enable multiple session testing (**-tcid base2**), the ipdiameter_cmd_alloc_access( ) function calls the following additional API functions:

- ipdiameter_set_avp_length( ): Sets the grouped Tunneling AVP length and adds the AVP.
- ipdiameter_grouped_avp_add_u32( ): Adds the Tunnel-Type AVP.
- ipdiameter_grouped_avp_add_u32( ): Adds the Tunnel-Medium-Type AVP.
- ipdiameter_grouped_avp_add_array( ): Adds the Tunnel-Client-Endpoint AVP.
- ipdiameter_grouped_avp_add_array( ): Adds the Tunnel-Server-Endpoint AVP.
- ipdiameter_get_avp_length( ): Adds the Tunneling AVP to the packet.
- ipdiameter_avp_add_array( ): Removes the AVP header.

The ipdiameter_cmd_opendiameter_access( ) function then calls ipdiameter_access_request( ) to handle the user request. This function performs validation checking on the user authentication request, creates an authentication session for the first packet in a session, selects the server associated with the session, and transfers the request to the authentication session's finite state machine to transmit the Diameter request to the Diameter server.

EAP Test Mode

The ipdiameter_cmd_opendiameter_access( ) function calls the following EAP-related functions:

- ipdiameter_cmd_alloc_eap_request( ): Manages AVPs for the EAP test modes.
- ipdiameter_eap_access_request( ): Sends the EAP request.
- ipdiameter_eap_free_der( ): Deallocates the EAP request data structure.

Parent topic: Developing a Diameter Application on page 66

Event Callback

You must implement a callback function in your application. To view an example implementation, see the ipdiameter_example_event_callback( ) function defined in the **ipdiameter_cmd_ipdiameterc.c** file. The ipdiameter_example_event_callback( ) function is invoked through a function pointer in the event callback field of the Ipdiameter_request structure. The event callback handles all incoming Diameter server reply packets, timeouts on access requests, and canceled requests.

The event callback function prototype is:

```
ipdiameter_example_event_callback(
    int event,
    struct Ipdiameter_request_struct *request_v,
```

```
    struct Ipdiameter_header *request_packet_v,
    struct Ipdiameter_header *response_packet_v,
    struct Ipdiameter_server *server_v);
```

The parameters are as follows:

event

Contains the event identifier that caused the event callback function to be called, see **ipdiameter_constant.h** for a list of callback events (**IPDIAMETER_EVENT_***).

request_v

A pointer to an **Ipdiameter_request** structure supplied by the application when it makes a request using the ipdiameter_access_request( ). For a definition of the structure, see **Ipdiameter_request_struct** in the **ipdiameter.h file**.

request_packet_v

A pointer to an **Ipdiameter_header** structure of an access request packet. For a definition of the structure, see **Ipdiameter_header_struct** in the **ipdiameter.h file**.

response_packet_v

A pointer to an **Ipdiameter_header** structure for a packet received from the server. For a definition of the structure, see **Ipdiameter_header_struct** in the **ipdiameter.h** file.

server_v

If the event callback function was called to handle a response from a Diameter server, **server_v** is a pointer to an **ipdiameter_server** structure that identifies the server. Otherwise, the value is **IP_NULL**. For a definition of the structure, see **ipdiameter_server_struct** in the **ipdiameter.h** file.

Parent topic: Developing a Diameter Application on page 66

Diameter EAP Access Request

The ipdiameter_eap_access_request( ) function implements the Diameter EAP authentication request and performs the following functions:

- validates the code and length fields from the passed-in EAP response packet
- allocates the EAP authentication cookie
- stores the EAP context and EAP callback function in the cookie
- fills the Diameter request structure
- formats the Diameter EAP Request (DER) packet
- submits the authentication request to the Diameter client

The Diameter EAP request structure is a function parameter passed in the ipdiameter_eap_access_request( ) function. For a definition of the Diameter EAP request structure, see **ipdiameter_eap_request_struct** in the **ipdiameter_eap.h** file. A Diameter EAP access request contains the following information:

- the EAP context associated with the request
- the EAP callback function
- the password information associated with the request
- the Diameter EAP Request (DER) data for the DER packet
- the server information used for the first request of a session

The context is saved in the EAP authentication cookie that is returned as a parameter in the EAP callback function. The EAP authenticator can retrieve any run-time context associated with this particular authentication request.

The DER contains the EAP response packet to be encapsulated in the EAP payload AVP.

Parent topic: Developing a Diameter Application on page 66

Diameter EAP Callback

The EAP callback function is invoked when the Diameter EAP receives an answer from the Diameter client. This callback function parses the callback events, records the Diameter answer result, and invokes the registered EAP callback function to reply to the EAP authenticator.

For an example implementation, see the ipdiameter_example_eap_callback( ) function in the **ipdiameter_cmd_ipdiameterc.c** file.

Parent topic: Developing a Diameter Application on page 66

6.2. Error Processing, Rerouting, and Failover

The following sections describe the error handling functions.

Undeliverable Packets on page 69
Retransmission Failure on page 69 All pending request messages are rerouted if retransmission failed. This event can occur if the agent does not receive a response message for the forwarded requests.
Rerouting on page 69 If a connection to a peer is not established, the agent moves request packets to a pending queue. Upon a transmission timeout, the agent then attempts to reroute the packets.
Failover on page 70 Upon transport layer failure or peer failure, rerouting is attempted for all packets. If rerouting fails, a packet with the **DIAMETER_UNABLE_TO_DELIVER** protocol error is sent to the previous peer.

Undeliverable Packets

If an agent receives messages indicating that a peer is unable to route a packet (with tagged protocol error **DIAMETER_UNABLE_TO_DELIVER**), it attempts to reroute the pending request. There is a single iteration of a routing table lookup. If a path is not found for a given destination, a packet with the **DIAMETER_UNABLE_TO_DELIVER** protocol error message is sent to the previous peer.

Parent topic: Error Processing, Rerouting, and Failover on page 69

Retransmission Failure

All pending request messages are rerouted if retransmission failed. This event can occur if the agent does not receive a response message for the forwarded requests.

Parent topic: Error Processing, Rerouting, and Failover on page 69

Rerouting

If a connection to a peer is not established, the agent moves request packets to a pending queue. Upon a transmission timeout, the agent then attempts to reroute the packets.

Parent topic: Error Processing, Rerouting, and Failover on page 69

Failover

Upon transport layer failure or peer failure, rerouting is attempted for all packets. If rerouting fails, a packet with the **DIAMETER_UNABLE_TO_DELIVER** protocol error is sent to the previous peer.

Parent topic: Error Processing, Rerouting, and Failover on page 69

6.3. Executing Shell Commands from an Application

The client and agent shell commands invoke C functions to implement command functionality. For example, the peeradd command from the client shell invokes the ipdiameter_cmd_routeadd( ) function from the **ipdiameter_cmd_diameterc.c** file. The peeradd command from the agent shell invokes the ipdiameter_cmd_routeadd( ) function from the **ipdiameter_cmd_diameterp.c** file. These functions are declared as **IP_STATIC int**. To access these functions as APIs, simply redefine the prototype as **IP_PUBLIC int**. For example, in the **ipdiameter_cmd_diameterp.c** file, change:

```
IP_STATIC int ipdiameter_cmd_peeradd(Vars *vars);
```

to

```
IP_PUBLIC int ipdiameter_cmd_peeradd(Vars *vars);
```

then change:

```
IP_STATIC int
ipdiameter_cmd_peeradd(Vars *vars)
{
...
```

to

```
IP_PUBLIC int
ipdiameter_cmd_peeradd(Vars *vars)
{
...
```

Recompile the source code and rebuild the VxWorks source to create a new VxWorks image. The Diameter Client Shell Commands and Associated APIs on page 70 table correlates the client shell commands to the newly defined APIs and the Diameter Agent Shell Commands and Associated APIs on page 71 table correlates the agent shell commands to the newly defined APIs. The **Vars** structure for the client is defined in **ipdiamater_cmd_diameterc.c** and the **Vars** structure for the agent is defined in **ipdiamater_cmd_diameterp.c**.

Table 1. Diameter Client Shell Commands and Associated APIs

Shell Command	API Routine
access	**ipdiameter_cmd_access(Vars *vars)**
exit	**ipdiameter_cmd_exit(Vars *vars)**
help	**ipdiameter_cmd_help(Vars *vars)**
nodeshow	**ipdiameter_cmd_nodeshow(Vars *vars)**

Shell Command	API Routine
openaccess	**ipdiameter_cmd_openaccess(Vars *vars)**
peeradd	**ipdiameter_cmd_peeradd(Vars *vars)**
peerdel	**ipdiameter_cmd_peerdel(Vars *vars)**
peermod	**ipdiameter_cmd_peermod(Vars *vars)**
peershow	**ipdiameter_cmd_peershow(Vars *vars)**
sessiondel	**ipdiameter_cmd_sessiondel(Vars *vars)**
sessionshow	**ipdiameter_cmd_sessionshow(Vars *vars)**
startc	**ipdiameter_cmd_startc(Vars *vars)**
stopc	**ipdiameter_cmd_stopc(Vars *vars)**

Table 2. Diameter Agent Shell Commands and Associated APIs

Shell Command	API Routine
access	**ipdiameter_cmd_access(Vars *vars)**
exit	**ipdiameter_cmd_exit(Vars *vars)**
help	**ipdiameter_cmd_help(Vars *vars)**
nodeshow	**ipdiameter_cmd_nodeshow(Vars *vars)**
openaccess	**ipdiameter_cmd_openaccess(Vars *vars)**
peeradd	**ipdiameter_cmd_peeradd(Vars *vars)**
peerdel	**ipdiameter_cmd_peerdel(Vars *vars)**
peermod	**ipdiameter_cmd_peermod(Vars *vars)**
peershow	**ipdiameter_cmd_peershow(Vars *vars)**
routeadd	**ipdiameter_cmd_routeadd(Vars *vars)**
routedel	**ipdiameter_cmd_routedel(Vars *vars)**
routeshow	**ipdiameter_cmd_routeshow(Vars *vars)**
policyadd	**ipdiameter_cmd_policyadd(Vars *vars)**

Shell Command	API Routine
policydel	**ipdiameter_cmd_policydel(Vars *vars)**
policyshow	**ipdiameter_cmd_policyshow(Vars *vars)**
sessiondel	**ipdiameter_cmd_sessiondel(Vars *vars)**
sessionshow	**ipdiameter_cmd_sessionshow(Vars *vars)**
startp	**ipdiameter_cmd_startp(Vars *vars)**
stopp	**ipdiameter_cmd_stopp(Vars *vars)**

7. Diameter Tutorial

7.1. Introduction

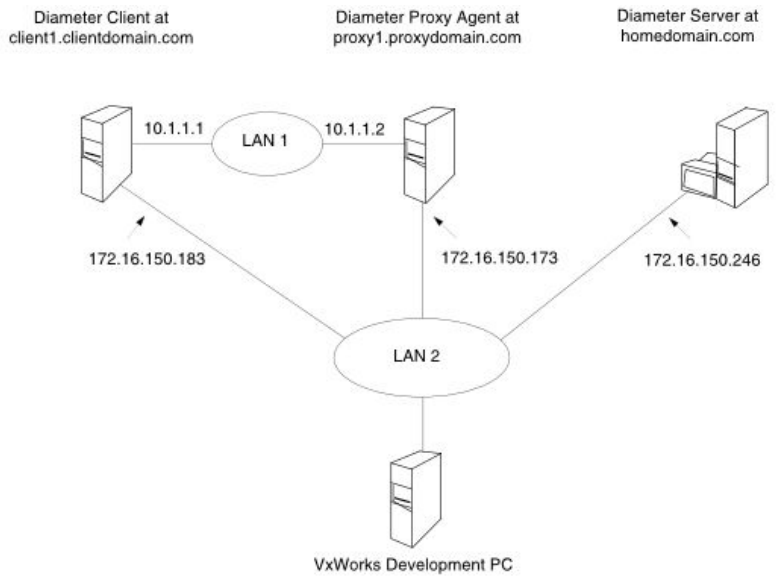
This tutorial describes the implementation of an example Diameter network environment with a Diameter client, a Diameter proxy agent, and an Open Diameter authentication server. The tutorial guides you through the creation and configuration of a Diameter client and a Diameter proxy agent followed by the testing of the Diameter protocol interfaces.

7.2. Diameter Example

This Diameter example involves the following:

- a VxWorks Diameter client running on a Core and configured with vxprj
- a VxWorks Diameter proxy agent running on a PowerPC and configured with Workbench
- a Diameter server with the Open Diameter server software.

The Diameter Example on page 73 figure details the Diameter configuration for the example project. The Diameter client is an access device for users at **clientdomain.com** and must be configured to relay user initiated Diameter messages to the Diameter proxy agent at **proxydomain.com**. The Diameter proxy agent must also be configured to relay the user initiated Diameter messages to the Diameter server at **homedomain.com**. The Diameter server is the home realm for **clientdomain.com** and acknowledges user Diameter requests and responds with answers. The answers are forwarded to the user over the Diameter proxy agent and the Diameter client connections.



Implementation Overview on page 73
Development Environment on page 74
Diameter Requirements Definition on page 75
Creating the Diameter Proxy Agent Image on page 76
Loading the Proxy Image onto the Target on page 77
Testing the Proxy Connection to the Diameter Server on page 79 From the proxy agent host shell, use the ping command to test the connection from the proxy agent to the Diameter server. For a Diameter server located at 172.16.150.246, use the following command:
Viewing the Proxy Agent Configuration on page 79 Once the proxy agent image is loaded onto the target, launch the Diameter agent shell.
Adding the Proxy Agent Coordinates to the Open Diameter Server on page 80 Add the proxy agent coordinates to the Open Diameter server configuration. Start the Open Diameter server application to initiate a dynamic peer connection to the proxy agent.
Viewing the Dynamically Configured Peer from the Proxy Agent on page 80 After launching the Open Diameter server application, verify that a peer entry has been automatically added to the Diameter proxy agent. To do this, access the Diameter agent shell and issue the peershow command. For example:
Creating the Diameter Client Image on page 80
Loading the Client Image onto the Target on page 83
Testing the Client Connection to the Diameter Proxy Agent on page 84 Use the ping command to test the network connection from the client to the Diameter proxy agent. For a Diameter proxy agent located at 10.1.1.2, use the following command:
Viewing the Client Configuration on page 84 Once the target image is loaded onto the target, launch the Diameter client shell.
Configuring the Proxy Agent Peer into the Client Peer Table on page 85 From the Diameter client shell, add a peer entry that defines the Diameter proxy agent. For example:
Testing the Diameter Protocol Interface to the Proxy Agent on page 86 Use the access shell command to test the Diameter connection to the proxy agent. For example:
Testing the Diameter Protocol Interface to the Server on page 87 To test the Diameter protocol interface from the client to the Diameter server via the proxy agent, use the openaccess command from the client shell. For example:
Retesting the Diameter Protocol Interface to the Server on page 87 To reinitiate the openaccess test from the Diameter client, complete the following steps:
Troubleshooting on page 88 To debug the operating environment, proceed as follows:

Implementation Overview

The implementation of the Diameter example involves the following:

- Development environment definition:
  - Development Environment on page 74
- Requirements definition for the Diameter client, proxy agent, and server:
  - Diameter Requirements Definition on page 75
- A Diameter Proxy Agent:
  - Creating the Diameter Proxy Agent Image on page 76
  - Loading the Proxy Image onto the Target on page 77
  - Testing the Proxy Connection to the Diameter Server on page 79
  - Viewing the Proxy Agent Configuration on page 79
  - Adding the Proxy Agent Coordinates to the Open Diameter Server on page 80
  - Viewing the Dynamically Configured Peer from the Proxy Agent on page 80
- A Diameter Client:
  - Creating the Diameter Client Image on page 80
  - Loading the Client Image onto the Target on page 83
  - Viewing the Client Configuration on page 84
  - Testing the Client Connection to the Diameter Proxy Agent on page 86
  - Configuring the Proxy Agent Peer into the Client Peer Table on page 85
- Diameter Testing:
  - Testing the Diameter Protocol Interface to the Proxy Agent on page 86
  - Testing the Diameter Protocol Interface to the Server on page 87
  - Retesting the Diameter Protocol Interface to the Server on page 87
- Troubleshooting the operating environment and the Diameter protocol:
  - Troubleshooting on page 88

Parent topic: Diameter Example on page 72

Development Environment

This section defines the development environment for the example. Development Environment for the Example on page 74 outlines the development workstation setup and the targets for the client and proxy agent.

Table 1. Development Environment for the Example

Client	Proxy Agent
VxWorks release: VxWorks 7 Compiler: LLVM Build tool: vxprj Project base directory: **C:\client1** Target image source directory: **C:\client1\default** Kernel shell script: **C:\rand\client1.txt** Target CPU: Core	VxWorks release: VxWorks 7 Compiler: GNU Build tool: Workbench Project base directory: *installDir* **/workspace/Proxy1** Target image source directory: *installDir* **/workspace/Proxy1/default** Kernel shell script: **C:\rand\proxy1.txt** Target CPU: 32-bit PowerPC

Client	Proxy Agent
Target IP address: 172.16.150.183 on gei0 Target Diameter address: 10.1.1.1 on gei1	Target IP address: 172.16.150.173 on dtsec0 Target Diameter address: 10.1.1.2 on dtsec0

The Diameter server is configured as follows:

- • Server application: Open Diameter version 1.0.7
  - Server IP address: 172.16.150.246
  - Proxy peer: proxy1.proxydomain.com
  - User: client1@clientdomain.com
  - Password: request
  - EAP MD5 secret: abcd1234

Information on Open Diameter is accessible from the following Web site:

http://diameter.sourceforge.net/

Parent topic: Diameter Example on page 72

Diameter Requirements Definition

The Requirements for the Example on page 75 table outlines the Diameter requirements for the client and the proxy agent.

Table 1. Requirements for the Example

Item	Client	Proxy Agent
Diameter node	Node type: Diameter client Host domain: IPv4 Diameter Identifier: client1.homedomain.com Host address: 10.1.1.1 Peer reconnection attempts: 3 Retransmit requests: 3 Peer reconnection interval: 30 seconds Host realm: homedomain.com Retransmit request interval: 10 seconds Host TCP port: 3868 Vendor ID: 123 Watchdog timeout: 30 seconds Session timeout: 30 Authorization grace period: 30 seconds Authentication lifetime: 360 seconds Authentication: EAP MD5 Application: Diameter EAP EAP MD5 Secret: abcd1234	Node type: Proxy agent Host domain: IPv4 Diameter Identifier: proxy1.homedomain.com Host address: 10.1.1.2 Peer reconnection attempts: 3 Retransmit requests: 3 Peer reconnection interval: 30 seconds Host realm: homedomain.com Retransmit request interval: 10 seconds Host TCP port: 3868 Vendor ID: 123 Watchdog timeout: 30 seconds
Command shell	Required	Required

Item	Client	Proxy Agent
Command shell test options	Required	Not applicable
Debugging options	None	None

Parent topic: Diameter Example on page 72

Creating the Diameter Proxy Agent Image

About This Task

Context for the current task.

Procedure

1. Create a new VSB project.

   To create a new VSB project, proceed as follows:

   1. Select **File > New > Project... >VxWorks 7 > VxWorks Source Build Project**
   2. In the Project name field, enter **MyPPC_BSP_VSB** and click Next.
   3. Select a PPC board support package and click **Finish**.

   The output files are created in *installDir* **/workspace/MyPPC_BSP_VSB**.

2. Include VSB options.

   To include the VSB options, proceed as follows:

   1. From the **Project Explorer**, access the **Source Build Configuration** tab.
   2.

      Set the following options to yes:
      - **Enable Diameter Proxy (COMPONENT_IPDIAMETERP)**
      - **Build both IPV4 and IPV6 libraries (FEATURE_IPNET_INET6)**

   3. Save the source build configuration.
   4. From the **Project Explorer**, select the **MyPPC_BSP_VSB** project and build the project.
   5. Observe the build progress from the **Build Console** tab.
3. Create a VIP.

   Base the VIP on the **MyPPC_BSP_VSB** VSB.

   To create a VIP from Workbench, proceed as follows:

   1. Select **File > New > Wind River Workbench Project**.
   2. Select **VxWorks Image Project** for the **Build Type**.
   3. In the **Project name** field, enter a **Proxy1** as the project name and click **Next**.

4. From the **Based on** drop-down list, select the **a source build project** option.
5. From the **Project** drop-down list, select **MyPPC_BSP_VSB**.
6. From the **Tool chain** drop-down list, select **gnu** and click **Finish**.

The output files are created in *installDir* **/workspace/Proxy1**.

4. Include components for the Proxy1 image build.

   Include the components first, then configure the parameter values for the example.

   Use Find to locate the following components and include them in your project.

   - **INCLUDE_IFCONFIG** to support the configuration of the kernel-resident network interfaces.
   - **INCLUDE_PING** to support the ping command.
   - **INCLUDE_IPATTACH** to support boot END connections to IPv4.
   - **INCLUDE_SHELL** to access the shell commands.
   - **INCLUDE_STARTUP_SCRIPT** to run the kernel shell script at startup.
   - **INCLUDE_SHELL_INTERP_CMD** to run the command interpreter for the kernel shell.
5. Build the Proxy1 VxWorks image project.

   Access the **Project Explorer** and click the **Proxy1** project to highlight the project item.

   Right-click the **Proxy1** project item and select **Build Project** to build the project.

   Observe the build progress from the **Build Console** tab. The output from the build image process is a set of files in *installDir* **/workspace/Proxy1/default**. Check that a **vxWorks** file is in the **default** directory.

Parent topic: Diameter Example on page 72

Loading the Proxy Image onto the Target

About This Task

Context for the current task.

Procedure

1. Make sure TFTP server is running on the workstation and is set to serve files from the following directory:

   c:\windriver\workspace\proxy1\default

2. Power up the target board into the U-Boot shell.
3. Set the target board network parameters from the U-Boot shell.

```
=> setenv ipaddr 172.16.150.173
=> setenv netmask 255.255.254.0
=> setenv serverip 172.16.144.165
=> setenv gatewayip 172.16.150.1
=> setenv bootargs 'dtsec(0,0)host:vxWorks h=172.16.144.165 e=172.16.150.173:fffffe00 g= 172.16
```

```
.150.1 u=target pw=vxTarget f=0x00'
=> saveenv
```

4. Boot and Run VxWorks.

   Load the VxWorks image and DTB files from the workstation using the U-Boot shell:

```
=> tftp 0x2000000 uVxWorks

=> tftp 0xf000000 p4080ds.dtb
```

   Run the VxWorks image using the U-Boot shell:

```
=> bootm 0x2000000 - 0xf000000
```

   Wait for the VxWorks shell prompt.

```
Target Name: vxTarget

                    VxWorks 7 SMP

                Copyright 1984-2018 Wind River Systems, Inc.

                    Core Kernel version: 1.2.7.0
                        Build date: Aug 10 2018 13:53:49
                                Board:
                            CPU Count: 8
                        OS Memory Size:
                    ED&R Policy Mode: Permanantly Deployed

        Adding 9870 symbols for standalone.
->
```

5. Attach the IP protocol to the corresponding device and configures the IP address.

```
-> ipAttach(1,"dtsec");
-> ifconfig "dtsec1 10.1.1.2 up"
```

   Check the IP address configurations:

```
-> ifconfig

dtsec0 Link type:Ethernet HWaddr 00:04:9f:03:0b:53
    capabilities: VLAN_MTU
    inet 172.16.150.173 mask 255.255.254.0 broadcast 172.16.151.255
...
dtsec1 Link type:Ethernet HWaddr 00:04:9f:03:0b:54
    capabilities: VLAN_MTU
    inet 10.1.1.2 mask 255.255.255.0 broadcast 10.10.10.255
...
```

Parent topic: Diameter Example on page 72

Testing the Proxy Connection to the Diameter Server

From the proxy agent host shell, use the ping command to test the connection from the proxy agent to the Diameter server. For a Diameter server located at 172.16.150.246, use the following command:

```
-> ping "172.16.150.246"


Pinging 172.16.150.246 (172.16.150.246) with 64 bytes of data:
Reply from 172.16.150.246 bytes=64 time=0ms ttl=64
Reply from 172.16.150.246 bytes=64 time=0ms ttl=64
Reply from 172.16.150.246 bytes=64 time=0ms ttl=64
Reply from 172.16.150.246 bytes=64 time=0ms ttl=64
```

Parent topic: Diameter Example on page 72

Viewing the Proxy Agent Configuration

Once the proxy agent image is loaded onto the target, launch the Diameter agent shell.

```
-> cmd diameterp
diameterp>
```

Use the show commands to view the Diameter proxy agent configuration. For example:

```
diameterp> nodeshow
State:                 opened and running
Node type:             Proxy
Node Addr:             10.1.1.2
Diameter Id:           proxy1.proxydomain.com
Realm Id:              proxydomain.com
Vendor Id:             0
TCP listening port:    3868
Supported application: (5)Diameter EAP

Timers:
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A

diameterp> routeshow
Diameter routes: None

diameterp> policyshow
Diameter proxy local policies: None
```

```
diameterp> peershow
Diameter Peers: None
diameterp>
```

Parent topic: Diameter Example on page 72

Adding the Proxy Agent Coordinates to the Open Diameter Server

Add the proxy agent coordinates to the Open Diameter server configuration. Start the Open Diameter server application to initiate a dynamic peer connection to the proxy agent.

Parent topic: Diameter Example on page 72

Viewing the Dynamically Configured Peer from the Proxy Agent

After launching the Open Diameter server application, verify that a peer entry has been automatically added to the Diameter proxy agent. To do this, access the Diameter agent shell and issue the peershow command. For example:

```
diameterp> peershow
Diameter Peers:                1
Diameter peer (821090):
  Peer Addr:                   172.16.150.246       (Server IP address )
  Peer Addr port:              2832                 (Port for non-Diameter packets )
  Peer Node Type:              Unknown
  Peer Realm:                  homedomain.com
  Peer Host name:              aaaserver.homedomain.com        (Server name )
  Peer is dynamic:             Yes                  (Peer dynamically discovered )
  Peer tls enabled:            No
  Peer state:                  R-Open               (Peer is connected )
  Peer fd:                     20
  Peer TLS connection:         No
  Peer supported application: (1)Diameter NASREQ
  Peer supported application: (2)Diameter Mobile-IP
  Peer supported application: (5)Diameter Diameter EAP
  Peer supported application: (3)Diameter Base Accounting
  Peer supported application: (4)Diameter Credit Control
```

Parent topic: Diameter Example on page 72

Creating the Diameter Client Image

About This Task

Procedure

1. Include the Diameter shell test functions.

   To enable testing of node connections to an Open Diameter server from the Diameter client shell, change **#undef OPEN_DIAMETER_TEST** to **#define OPEN_DIAMETER_TEST** in the following source files:

- **ipdiameter_client.c**
- **ipdiameter_cmd_diameterc.c**
  ipdiameter_rx.c
- **ipdiameter_tx.c**

2. Include the Diameter EAP test functions.

   To enable the testing of Diameter EAP, change #undef **OPEN_DIAMETER_EAP** to #define **OPEN_DIAMETER_EAP** in **ipdiameter_cmd_diameterc.c**.

3. Create the VSB.
   1. To create a new VSB project, access the **c:\client1** directory and use the vsb create command from the VxWorks development shell.

      ```
      > vxprj vsb create -bsp itl_generic
      ```

      A new folder is created in the **installDir/vxworks-7/workspace/vsb_itl_generic_2_0_0_0** directory.
   2. Scroll down the VSB option list using the **Enter** key, option-by-option, and set the following VSB options to **yes**:
      - Enable OpenSSL (OPENSSL)
      - Enable Diameter Client (COMPONENT_IPDIAMETERC)
      - Build both IPV4 and IPV6 libraries (FEATURE_IPNET_INET6)
      - Enable EAP (IPNET_EAP)
      - Enable IPSEC (IPNET_IPSECIKE)
   3. Scroll to the end of the configuration options.
   4. Access the VSB project directory where the **.wpj** project file is located, and run the **make** command.

      ```
      > make
      ```

4. Create a VIP.

   From a VxWorks development shell, use the following create command to create the VIP:

   ```
   > vxprj create itl_generic llvm -vsb vsb_itl_generic_2_0_0_0 client1
   ```

   Check that the **client1.wpj** project file is in **c:\client1**.

5. Include Diameter client components using vxprj.

   Use the following vxprj commands to include Diameter client components in the project. Execute these vxprj commands from the project directory where the project file (**client1.wpj**) is located.

   ```
   > vxprj component add INCLUDE_IPDIAMETERC

   > vxprj component add INCLUDE_IPDIAMETERC_CMD
   ```

   Use the following vxprj commands to include support components in the project. Additional components added automatically by vxprj are listed in the command output.

   ```
   > vxprj component add INCLUDE_PING
   > vxprj component add INCLUDE_IFCONFIG
   > vxprj component add INCLUDE_IPATTACH
   > vxprj component add INCLUDE_USE_NATIVE_SHELL
   ```

```
> vxprj component add INCLUDE_STARTUP_SCRIPT
> vxprj component add INCLUDE_IPEAP
```

To view all the components in the project, use the vxprj component list command.

6. Configure Diameter client parameters using vxprj.
   1. To view the current configuration use the vxprj parameter value command.

```
> vxprj parameter value DIAMETER

DIAMETER_APPLICATION_LIST = "2"
DIAMETER_ASR_COMPLY = "1"
DIAMETER_AUTH_GRACE_PERIOD = "30"
DIAMETER_AUTH_LIFETIME = "360"
DIAMETER_HOST_DOMAIN = "2"
DIAMETER_HOST_IPV4_ADDRESS = "10.1.2.1"
DIAMETER_HOST_IPV6_ADDRESS = "3ffe:300::5"
DIAMETER_HOST_REALM = "homedomain.com"
DIAMETER_HOST_TCP_PORT = "3868"
DIAMETER_IDENTIFIER = "test.homedomain.com"
DIAMETER_PEER_MAX_RECONN_ATTEMPT = "30"
DIAMETER_PEER_RECONN_INTERVAL = "30"
DIAMETER_REQ_MAX_RETX = "3"
DIAMETER_REQ_RETX_INTERVAL = "10"
DIAMETER_SESSION_TIMEOUT = "30"
DIAMETER_TLS_CERTIFICATE = "certfile.pem"
DIAMETER_TLS_ENABLED = "0"
DIAMETER_TLS_KEY_FILE = "keyfile.pem"
DIAMETER_TLS_KEY_PASSWORD = "clientkey"
DIAMETER_TLS_ROOT_CERTIFICATE = "cacertfile.pem"
DIAMETER_TLS_ROOT_DIR = "/romfs"
DIAMETER_VENDOR_ID = "0"
DIAMETER_WATCHDOG_TIMEOUT = "30"
```

   2. Use the following list of vxprj commands to set parameter values for the tutorial. Execute these commands from the project directory where the project file (**client1.wpj**) is located.

```
> vxprj parameter setstring DIAMETER_HOST_IPV4_ADDRESS "10.1.1.1"
> vxprj parameter setstring DIAMETER_HOST_REALM "clientdomain.com"
> vxprj parameter setstring DIAMETER_IDENTIFIER "client.clientdomain.com"
> vxprj parameter setstring DIAMETER_APPLICATION_LIST "5"
```

   3. Check the new configuration with the following command:

```
> vxprj parameter value DIAMETER
```

   4. Set key encrypting password to a random string of at least 40 characters, containing both lower and upper case, numbers and special characters.

```
> vxprj parameter setstring SEC_VAULT_KEY_ENCRYPTING_PW "1azCDRTYUIOP1234GHJKL;qwer567890ZXC
VBNM,./97531"
```

7. Configure EAP MD5 parameters using vxprj.
   1. To view the current configuration use the vxprj parameter value command.

```
> vxprj parameter value EAP_MD5

EAP_MD5 = "0"
EAP_MD5_SECRET = "this_should_be_secret"
```

2. Use the following list of vxprj commands to set parameter values for the tutorial.

```
> vxprj parameter setstring EAP_MD5 "1"
> vxprj parameter setstring EAP_MD5_SECRET "abcd1234"
```

3. Check the new configuration with the following command:

```
> vxprj parameter value EAP_MD5
```

8. Define the parameters for SEC_VAULT_KEY_ENCRYPTING_PW. This value is a random string of at least 40 characters containing both lower and upper case, numbers and special characters.

```
> vxprj parameter setstring SEC_VAULT_KEY_ENCRYPTING_PW "<key password>"
```

9. Build the Diameter client image using vxprj.

   To build the project, run the vxprj build command from the project directory where the **client1.wpj** project file is located.

```
> vxprj build
```

   Check that a default directory is in **c:\client1** and a **VxWorks** executable file is in the **default** directory.

   The **vxWorks.exe** file contains the Diameter client image for the Core target.

Parent topic: Diameter Example on page 72

Loading the Client Image onto the Target

About This Task

Procedure

1. Configure the target boot parameters.

   Configure the target boot parameters to load the **C:\client1\vxWorks.exe** client image file and run the startup script on startup. For example:

```
[VxWorks Boot]: c

boot device          : gei
unit number          : 0
processor number     : 0
file name            : c:\client1\vxworks
inet on ethernet (e) : 172.16.150.183:ffffe00
host inet (h)        : 172.16.144.165
gateway inet (g)     : 172.16.150.1
user (u)             : proxy1_client1
```

```
ftp password (pw)     : proxy1_client1
flags (f)             : 0x0
startup script (s)    : c:\client1\client1.txt
```

Set the **user** and **ftp password** to **proxy1_client1**. These are your login credentials for the FTP server configured in Loading the Proxy Image onto the Target on page 77. You can use the same FTP server to load the client files into the target.

The startup script, **client1.txt**, runs at boot up. The script attaches the IP protocol to the corresponding device and configures the IP address. For example:

```
ipAttach(1,"gei");
ifconfig "gei1 10.1.1.1 up";
```

2. Restart the target.

   From the VxWorks shell, restart the target and wait for the VxWorks shell prompt.

```
[VxWorks Boot]: @
...
Executing startup script 'c:\client1\client1.txt'...
...
gei0    Link type:Ethernet  HWaddr 00:02:b3:24:84:9d  Queue:none
        capabilities: VLAN_MTU
        inet 172.16.150.183  mask 255.255.254.0  broadcast 172.16.151.255
...
gei1    Link type:Ethernet  HWaddr 00:d0:b7:5d:af:03  Queue:none

        capabilities: VLAN_MTU

        inet 10.1.1.1  mask 255.0.0.0  broadcast 10.255.255.255
==========
```

Parent topic: Diameter Example on page 72

Testing the Client Connection to the Diameter Proxy Agent

Use the ping command to test the network connection from the client to the Diameter proxy agent. For a Diameter proxy agent located at 10.1.1.2, use the following command:

```
-> ping "10.1.1.2"

Pinging 10.1.1.2 (10.1.1.2) with 64 bytes of data:
Reply from 10.1.1.2 bytes=64 time=0ms ttl=64
Reply from 10.1.1.2 bytes=64 time=0ms ttl=64
Reply from 10.1.1.2 bytes=64 time=0ms ttl=64
Reply from 10.1.1.2 bytes=64 time=0ms ttl=64
```

Parent topic: Diameter Example on page 72

Viewing the Client Configuration

Once the target image is loaded onto the target, launch the Diameter client shell.

```
-> cmd diameterc
diameterc>
```

Use the show commands to view the Diameter client configuration. For example:

```
diameterc> nodeshow
State:                  opened and running
Node type:              Client
Node Addr:              10.1.1.1
Diameter Id:            client.clientdomain.com
Realm Id:               clientdomain.com
Vendor Id:              0
TCP listening port:     3868
Supported application: (5)Diameter EAP

Timers:
  Session timeout:            30
  Authorization lifetime:     360
  Auth Grace Period:          30
  Peer reconnection interval: 30
  Max reconnection attemps:   30
  Watchdog timeout:           30
  Request retx interval:      10
  Max request retx:           3

TLS:
  TLS enabled:                No
  Node certficate file:       N/A
  Node private key file:      N/A
  Node key file's key passwd: N/A
  CA root certificate file:   N/A
  Certificate root directory: N/A

diameterc> peershow
Diameter Peers: None

diameterc> sessionshow
Diameter Sessions: None
```

Parent topic: Diameter Example on page 72

Configuring the Proxy Agent Peer into the Client Peer Table

From the Diameter client shell, add a peer entry that defines the Diameter proxy agent. For example:

```
diameterc> peeradd -peerip 10.1.1.2 -realm proxydomain.com -port 3868 -tlsenabled 0 -peertype 7
```

View the new entry.

```
diameterc> peershow
Diameter Peers:        1
Diameter peer (f93e90):
  Peer Addr:           10.1.1.2
  Peer Addr port:      3868
```

```
  Peer Node Type:      Proxy
  Peer Realm:          proxydomain.com
  Peer Host name:      10.1.1.2
  Peer is dynamic:     No
  Peer tls enabled:    No
  Peer state:          Closed
  Peer fd:             -1
  Peer TLS connection: No
```

Parent topic: Diameter Example on page 72

Testing the Diameter Protocol Interface to the Proxy Agent

Use the access shell command to test the Diameter connection to the proxy agent. For example:

```
diameterc> access -peerip 10.1.1.2 -user user@proxydomain.com -pwd request -realm homedomain.com
```

From the client command shell, observe the response. For example:

```
diameterc> ipdiameter_example_event_callback: rx event access_timeout
Diameter request (f94388):
  Request cookie:          102e940
  Request unique:          2700748970
  Request session ID:      client.clientdomain.com;3;0;0
  Request appl ID:         0
  Request pwd_type:        1
  Request pwd:             request
  Request chap_ident:      1
  Selected server index:   -1
  Request server_addr:     <unknown sa_family>
  Request server_type:     Full Server
```

The reception of an **access_timeout** event is normal as the proxy agent cannot respond to the access request.

From the agent command shell, observe that a dynamically configured route for the client is registered in the peer table. For example:

```
diameterp> peershow
Diameter Peers:                2
Diameter peer (8aeec0):
  Peer Addr:                   10.1.1.1     (Peer address should be that of the client)
  Peer Addr port:              49155
  Peer Node Type:              Unknown
  Peer Realm:                  clientdomain.com
  Peer Host name:              client.clientdomain.com
  Peer is dynamic:             Yes         (Peer is dynamic)
  Peer tls enabled:            No
  Peer state:                  R-Open       (Peer state should be R-Open)
  Peer fd:                     19
  Peer TLS connection:         No
  Peer supported application:  (5)Diameter EAP

Diameter peer (821090):
  Peer Addr:                   172.16.150.246
```

```
  Peer Addr port:              2832
  Peer Node Type:              Unknown
  Peer Realm:                  homedomain.com
  Peer Host name:              aaaserver.homedomain.com
  Peer is dynamic:             Yes
  Peer tls enabled:            No
  Peer state:                  R-Open
  Peer fd:                     20
  Peer TLS connection:         No
  Peer supported application: (1)Diameter NASREQ
  Peer supported application: (2)Diameter Mobile-IP
  Peer supported application: (5)Diameter EAP
  Peer supported application: (3)Diameter Base Accounting
  Peer supported application: (4)Diameter Credit Control
```

Parent topic: Diameter Example on page 72

Testing the Diameter Protocol Interface to the Server

To test the Diameter protocol interface from the client to the Diameter server via the proxy agent, use the openaccess command from the client shell. For example:

```
diameterc> openaccess -user user@clientdomain.com -pwd request -servip 172.16.150.246 -realm homed
omain.com -tcid eap -rar 1
```

The openaccess command selects the test mode (-tcid eap) and creates two sessions (Diameter requests) and terminates both sessions.

The response from the Diameter server through the Diameter proxy agent should be similar to the following output:

```
diameterc> ipdiameter_example_eap_callback: event 1
ipdiameter_example_eap_callback: EAP request(session client.clientdomain.com.clientdomain.com;1231
443877;197886)
ipdiameter_example_eap_callback: event 1
ipdiameter_example_eap_callback: EAP request(session client.clientdomain.com.clientdomain.com;1231
443877;197887)
ipdiameter_example_eap_callback: event 1
ipdiameter_example_eap_callback: EAP auth success(session client.clientdomain.com.clientdomain.com
;1231443877;197886)
ipdiameter_example_eap_callback: event 1
ipdiameter_example_eap_callback: EAP auth success(session client.clientdomain.com.clientdomain.com
;1231443877;197887)
```

Parent topic: Diameter Example on page 72

Retesting the Diameter Protocol Interface to the Server

To reinitiate the openaccess test from the Diameter client, complete the following steps:

1. Exit the Diameter server application.
2. Launch the Diameter server application.
3. Check that a dynamic peer entry has been added to Diameter proxy agent peer table. Enter peershow from the Diameter agent command shell.

4. Exit the Diameter client shell.
5. Restart the Diameter client daemon target from the VxWorks C shell by entering reboot.
6. Launch the client shell from the VxWorks C shell.
7. Add the proxy agent peer coordinates. Use peeradd from the Diameter client shell.
8. Run the openaccess command from the Diameter client.
9. Verify the results, see Testing the Diameter Protocol Interface to the Server on page 87.

Parent topic: Diameter Example on page 72

Troubleshooting

To debug the operating environment, proceed as follows:

- Check that the client target can ping the proxy target.
- Check that the proxy target can ping the server.
- Check the cable connections, LAN configuration, and the gateway configuration.
- Check that the necessary components are included in the image. Use the Workbench Kernel Configuration tool or the vxprj component list command to view the image components.
- Check that the necessary parameters are configured for the given scenario. Use the Workbench Kernel Configuration editor or the vxprj parameter value command to view the component parameter values.

To debug the Diameter protocol, you can enable debug options for the client, the proxy agent, or both. For configuration details, see Configuring the Diameter Debug Options on page 31. After selecting the debug options, you rebuild the VSB, rebuild the VIP, and reload the image into the target. Observe the debug output on the target console. Run the access shell command from the client to debug client to proxy agent Diameter protocol problems. Run the openaccess shell command from the client to debug client to server Diameter protocol problems.

> Note: Enabling debug options adds printf commands to the code that can change the timing behavior and introduce delays that may impact functionality.

You can also change the system logging level for Diameter (**ipdiameter**).

Parent topic: Diameter Example on page 72

3. APPENDICES

1. Openssl Example

1.1. Introduction

This appendix provides instruction on how to generate certificate and key files that are suitable for testing the Diameter TLS configuration and are not meant for any other purpose. These instructions have been tested on a Linux machine, which has OpenSSL installed.

For more information regarding SSL, see the VxWorks 7 SSL Programmer's Guide.

1.2. Self-signed Certificate and Private Key

Use the following command to generate a self-signed Configuration Authority certificate file called **cacertfile.pem** and a private key file called **cakey.pem**. The private key file is used to create digitally signed user certificates from Certificate Signing Request files.

```
openssl req -new -x509 -out cacertfile.pem -outform PEM -days 3650 -config cacert.cnf -keyout cake
y.pem -extensions v3_ca
```

This command uses information from an openssl configuration file called **cacert.cnf**, which is passed to the command using the -config option. The contents of this file is as follows:

```
prompt = no
[ req ]
default_bits = 1024 # size of keys
distinguished_name = req_distinguished_name
[ req_distinguished_name ]
C = CA
ST = Ontario
L = Ottawa
O = Wind River Systems
OU = Tech Pubs
CN = yow-techpubs
emailAddress = tech.writer@windriver.com
[ v3_ca ]
basicConstraints = CA:TRUE
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer:always
```

The output of the command follows. The user is prompted to enter a pass phrase twice.

```
Generating a 1024 bit RSA private key
writing new private key to 'cakey.pem'
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```

Query the Certificate Authority, self-signed certificate using the following commands:

```
openssl x509 -in cacert.pem -noout -text
openssl x509 -in cacert.pem -noout -dates
openssl x509 -in cacert.pem -noout -purpose
```

1.3. Certificate Signing Request and Private Key

Use the following command to create a Configuration Signing Request (CSR) file called **userreq.pem** and a private key file called **keyfile.pem**.

```
openssl req -new -out userreq.pem -config userreq.cnf -keyout keyfile.pem
```

This command uses information from an openssl configuration file called **userreq.cnf,** which is passed to the command using the **-config** option. The contents of userreq.cnf follows:

```
prompt = no
[ req ]
default_bits = 1024 # size of keys
distinguished_name = req_distinguished_name
req_extensions = v3_req
[ req_distinguished_name ]
C = CA
ST = Ontario
L = Ottawa
O = Wind River Systems
OU = User's Organization
CN = yow-server
emailAddress = postmaster@windriver.com
[ v3_req ]
basicConstraints = CA:FALSE
subjectKeyIdentifier = hash
```

The command to verify the output of the CSR creation operation follows:

```
openssl req -in userreq.pem -text -verify -noout
```

1.4. Signed Certificate Created from a Certificate Signing Request

Before processing the certificate signing request do the following:

1. Make a directory to contain signed certificates using the command mkdir newcerts
2. Make a directory to contain the Certificate Authority's private key files and put the key file there using the commands mkdir private and mv cakey.pem private
3. Create a file that records the creation of signed certificates using the command touch index.txt
4. Create a file that is used to number the signed certificates stored in the **newcerts** directory using the command echo '01' > serial

Use the following command to create a signed certificate file called **usercert.pem** follows:

```
openssl ca -out certfile.pem -config ./usercert.cnf -infiles
    userreq.pem
```

The contents of usercert.cnf follows:

```
dir = .
prompt = no
[ ca ]
default_ca = CA_default
```

```
[ CA_default ]
serial = $dir/serial
database = $dir/index.txt
new_certs_dir = $dir/newcerts
certificate = $dir/cacertfile.pem
private_key = $dir/private/cakey.pem
default_days = 365
default_md = md5
preserve = no
email_in_dn = no
nameopt = default_ca
certopt = default_ca
policy = policy_match
[ policy_match ]
countryName = match
stateOrProvinceName = match
organizationName = match
organizationalUnitName = optional
commonName = supplied
emailAddress = optional

[ req ]
default_bits = 1024 # size of keys
distinguished_name = req_distinguished_name
[ req_distinguished_name ]
C = CA
ST = Ontario
L = Ottawa
O = Wind River Systems
OU = User's Organization
CN = yow-server
emailAddress = postmaster@windriver.com
```

The output from this command follows:

```
Using configuration from ./usercert.cnf
Enter pass phrase for ./private/cakey.pem:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'CA'
stateOrProvinceName   :PRINTABLE:'Ontario'
localityName          :PRINTABLE:'Ottawa'
organizationName      :PRINTABLE:'Wind River Systems'
organizationalUnitName:PRINTABLE:'Users Organization'
commonName            :PRINTABLE:'yow-server'
Certificate is to be certified until Nov  9 16:15:58 2011 GMT (365 days)
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
```

Inspect the signed certificate using the following command:

```
openssl x509 -in certfile.pem -noout -text -purpose | more
```

2. Using ROMFS to Store Certifications

2.1. Introduction

This appendix provides instruction on how to give the Diameter client or proxy agent access to the certificate and key files that it needs for TLS operations.

2.2. Using a ROM File System

To use ROMFS to access the certificate and key files used in TLS operations, do the following in Workbench.

> Note: The procedural description assumes that you have already created the certificate and key files as described in Openssl Example on page 89. The names of the certificate and key files agree with the defaults given to the Diameter TLS parameters that reference them.

1. In the Workbench Project Explorer select the client or proxy VIP so that the new ROMFS project will be come a subproject of the VIP
2. Select **File > New > Wind River Workbench Project**.
3. In the text box labeled **Project name**, enter the name of the project.
4. Leave Create project in workspace selected, and click **Finish**.
5. The new ROMFS project appears under the diameter VIP project in the Project Explorer, and the File System Contents editor opens.
6. In the File System Contents editor click **Add External**.
7. Using the native file browser dialog, browse to the folder containing the certificate and key files.
8. If the VIP includes the secure key storage components then select **cacertfile.pem** and **certfile.pem** and click **Open**.
9. If the VIP does not include the secure key storage components then select **cacertfile.pem**, **certfile.pem**, and **keyfile.pem** and click **Open**.
10. Save the ROMFS project.
11. Rebuild the Diameter VIP.
12. Observe the build console output. Notice that the files that you selected from the host directory for inclusion in the ROMS file system are copied into the Contents folder, which is a subfolder of the romfs project folder. This copy operation is done in every subsequent build or rebuild of the Diameter VIP.

3. Glossary

AAA

Authentication, Authorization, and Accounting

accounting request

A client-to-server request for session accounting information.

access request message

A client-to-server request for authentication on behalf of a user.

API

Application Programming Interface

AVP

Attribute Value Pair

CA

Certificate of Authenticity

CCI

Common Cryptographic Interface

CHAP

Challenge-Handshake Authentication Protocol

DEA

Diameter Extensible Authentication Protocol Answer

DER

Diameter Extensible Authentication Protocol Request

EAP

Extensible Authentication Protocol

FSM

Finite State Machine

HA

Home Agent

IAS

Internet Authentication Service

IETF

Internet Engineering Task Force

IKE

Internet KEY Exchange

IP

Internet Protocol

IPsec

Internet Protocol Security

MD5

Message Digest Algorithm support from CCI

MIB

Management Information Base

NAS

Network Access Server

PAP

Password Authentication Protocol

PPP

Point-to-Point Protocol

RADIUS

Remote Authentication Dial In User Service

RADIUS Server

A server that support the RADIUS protocol.

RTP

Real-Time Process

SCTP

Stream Control Transmission Protocol

SSL

Secure Sockets Layer

TCP

Transport Control Protocol

TLS

Transport Layer Security

UPD

User Datagram Protocol